

INDIAN INSTITUTE OF TECHNOLOGY BOMBAY

DUAL DEGREE THESIS PROJECT

Robust Quantum Optimal Control Machine Learning Approach

Author:
MUDE CHAITHANYA NAIK

Supervisor:
Prof. Sai VINJANAMPATHY

Submitted in partial fulfillment of the requirements of the degree of Master of Technology

Quantum Computing
Computer Science and Engineering
Indian Institute of Technology Bombay



June 30, 2021

Declaration of Authorship

I, MUDE CHAITHANYA NAIK, declare that this thesis titled, “Robust Quantum Optimal Control - Machine Learning Approach” and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signature:

Name of the student: Chaithanya Naik Mude

Roll Number: 160050102

Date: 30 June 2021

This thesis entitled **Robust Quantum Optimal Control - Machine Learning Approach** by **MUDE CHAITHANYA NAIK** is approved for the degree of Master of Technology.

Examiners

Prof. Ashutosh Gupta

Supervisor(s)

Prof. Sai Vinjanampathy

Prof. G. Siva Kumar

Chairman

Prof. Bhaskaran Muralidharan

Date: 28 June 2021

Place: IIT Bombay

INDIAN INSTITUTE OF TECHNOLOGY BOMBAY

Abstract

Prof. Sai Vinjanampathy
Computer Science and Engineering

Robust Quantum Optimal Control - Machine Learning Approach

by MUDE CHAITHANYA NAIK

The ability to utilize quantum algorithms to solve computationally expensive problems relies on the ability to engineer high-fidelity quantum gates on quantum processors in presence of system noise for fault-tolerant quantum computation. Quantum optimal control methods have been employed widely to realize high-fidelity quantum gates. We discuss about the similarity between the quantum control problem with that of sequence to sequence learning problems along with ability of reinforcement learning approach to perform good even in presence of perturbations. We also discuss about the possibility of utilizing search based methods like Grover's search and bi-directional search. We also discuss about how to scale the solution for higher qubit system and issues associated with the same. We employ deep reinforcement learning methods, trust region policy optimization (TRPO) and Proximal policy optimization (PPO) and deep learning methods based on encoder-decoder architecture using Long Short Term Memory (LSTM) units. We apply the former on single qubit, two qubit and three qubit systems to achieve infidelity of the order 10^{-3} and latter on two qubit cross resonance system to achieve infidelity of the order 10^{-5} .

Acknowledgements

I thank my advisor, Prof. Sai Vinjanampathy for guiding me, in understanding quantum computation and quantum control at a deeper level and for the very interesting discussions on various topics throughout my thesis to gain broader understanding on the field. I want to thank my Faculty advisor, Prof. Siva Kumar, for helping me in smoothly completing my inter-departmental research project.

Contents

Declaration of Authorship	iii
Abstract	vii
Acknowledgements	ix
Contents	xi
List of Figures	xiii
1 Introduction	1
1.1 Overview	1
1.2 Previous Work	2
2 Quantum Computing	3
2.1 Quantum Computing	3
2.1.1 Bits and Qubits	3
2.1.2 Quantum Mechanics	4
2.2 Classical Gates and Quantum Gates	5
2.3 Quantum Gates and their Decomposition	5
2.4 Quantum Control	6
2.4.1 Forward and Inverse Hamiltonian Problem	6
2.4.2 From Control pulses to Unitaries	6
Transmon system	6
3 Machine Learning	9
3.1 Formalism	9
3.1.1 Supervised learning	9
3.1.2 Un-Supervised learning	9
3.1.3 Reinforcement learning	9
3.1.4 Learning workflow	9
3.2 Artificial Neural Networks	10
3.3 Different types of networks	11
3.3.1 Feed-Forward Neural Network	11
3.3.2 Convolutional Neural Network	12
3.3.3 Recurrent Neural Network	12
3.3.4 Auto-Encoders	13
3.4 Natural Language Processing	14
3.5 Sequence to sequence models	15
3.6 Control problem as seq2seq problem	15
4 Reinforcement Learning	17
4.1 Formalism	17
4.2 Q-learning	18
4.3 Policy Gradient methods	19
4.3.1 ACTOR-CRITIC	19
4.3.2 Trust Region Policy Optimization	20
4.3.3 Proximal Policy Optimization	21

4.3.4	Reinforcement learning for temporal data	21
5	Methods	23
5.1	Differential Evolution for Quantum Control	23
5.1.1	Differential Evolution Overview	23
5.1.2	SUSSADE	23
5.2	Push Pull optimization	24
5.3	RNNs - sequence to sequence learning	25
5.4	Search based methods	25
5.4.1	Grover's search	25
5.4.2	Bi-directional search	26
5.5	Deep Reinforcement learning for quantum control	27
5.6	Implementation Details	27
5.7	Physical Systems	28
5.7.1	Cross-Resonance Gate	28
5.7.2	Single-qubit	28
5.7.3	Two-qubit	29
5.7.4	three-qubit	29
6	Results	31
6.1	Simulation results for single-qubit	31
6.1.1	Comparison with results in paper by Marin Bukov et. al	31
6.1.2	Proximal Policy Optimization	31
6.1.3	Trust Region Policy Optimization	32
6.1.4	Variation across length of pulse sequence	32
6.2	Simulation results for two-qubit	33
6.2.1	Comparison with results in paper by Zheng An et. al	33
6.2.2	Proximal Policy Optimization	33
6.2.3	Trust Region Policy Optimization	34
6.3	Simulation results for three-qubit	34
6.3.1	Comparison with results in paper by Barry Sanders et. al	34
6.3.2	Proximal Policy Optimization	34
6.3.3	Trust Region Policy Optimization	35
6.4	Simulation results for two-qubit Cross Resonance Gate	36
6.4.1	RNN method	36
7	Discussion about scalability and performance	39
7.1	Comparison of methods	39
7.2	Performance & how it varies	39
7.3	Scalability	40
8	Conclusions and Future work	41
8.1	Extending to higher qubit systems	41
8.2	Utilizing the solution to build scalable practical machines	41
A	Software tool-kit	43
A.1	Design of the model	43
A.1.1	Components	43
A.2	Implementation details	44

List of Figures

2.1	Representing quantum state $ \psi\rangle$ using bloch sphere	4
3.1	ML workflow	10
3.2	Artificial Neural networks and Deep Neural Networks	11
3.3	Feed Forward Neural Network	11
3.4	CNN unit 3-D representation and mapping of units between layers	12
3.5	Convolutional Neural Network using digit recognition example	12
3.6	Recurrent Neural Network	13
3.7	Auto-encoder	13
3.8	Hopfield, Helmholtz networks and Boltzmann machine	14
4.1	Basic intuition for actor critic models	19
4.2	The workflow of actor critic models	20
5.1	Outline of Grover's Search	25
5.2	Grover Iteration	26
6.1	Proximal Policy Optimization for single qubit system; Maximum Fidelity: 0.9993	31
6.2	Trust Region Policy Optimization for single qubit system; Maximum Fidelity: 0.99987	32
6.3	Variation of average and maximum fidelity with respect to length of the pulse sequence	32
6.4	Proximal Policy Optimization for two qubit system; Maximum Fidelity: 0.9933	33
6.5	Trust Region Policy Optimization for two qubit system; Maximum Fidelity: 0.9993	34
6.6	Proximal Region Policy Optimization for three qubit system; Maximum Fidelity: 0.9927	35
6.7	Trust Region Policy Optimization for three qubit system; Maximum Fidelity: 0.99977	35
6.8	Three qubit system; Toffoli target gate; Maximum Fidelity: 0.9993	36
6.9	Three qubit system; Fredkin target gate; Maximum Fidelity: 0.9995	36
6.10	Amplitudes of two qubit cross resonance system; Maximum fidelity: 0.99996	37
6.11	Phases of two qubit cross resonance system; Maximum fidelity: 0.99996	37
6.12	State Transition	38
6.13	Loss value for QFT target gate	38
A.1	Outline of modular units with direction of flow of the data	43

I dedicate my dissertation work to my family and my friends. A special feeling of gratitude to my loving parents, Dr. Ramamurthy Naik and Lakshmi, who had supported me during this uncertain time of pandemic. My sister, Dr. Praneetha, never left my side, constantly motivating me throughout this duration and is very special.

Chapter 1

Introduction

1.1 Overview

Moore's Law states that we can expect the speed and capability of our computers to increase every couple of years, and we will pay less for them. The most profound observation and prediction made on Integrated Chips by Gordon Moore for classical computation had continued since 1970. Given the recent advancement in the quantum computation and quantum information theory, Hartmut Neven, director of Google AI lab, has stated that the growth in power with each new improvement to Google's best quantum processor is growing at not just an exponential rate, like in Moore's Law, but at a doubly-exponential rate.

The computational power after a particular n -qubit quantum computer would be so huge, that the computation of previously unimaginable problems that can be performed on a cluster of super-computers can be performed using a quantum computer. Being said that, quantum algorithms need a quantum computation model implemented using quantum circuits. The critical step that converts theory into reality is fault-tolerant, highly efficient design of quantum controls to translate the given algorithm into a set of analog control signals that can be interpreted by quantum computers around the Hilbert space.

The control procedure to get the precise quantum gate controls decides the fidelity and speed of each quantum operation. The above metrics are used to determine the efficiency of the control protocol. Higher gate fidelity reduces the computational overhead for fault-tolerant error correction and smaller run-time provide the flexibility to implement deeper quantum circuits thereby allowing us to implement complicated quantum algorithms.

The fault-tolerant quantum computations should be performed for obtaining best results out of quantum algorithms which require high fidelity quantum gates. Another important detail that determines the practical application is the scalability of the approach, which should be able to solve the problem for any quantum gate. In classical computation, a universal gate set is used to implement any general classical circuit. A similar approach which is discussed in the next chapter provides the details about a similar approach for quantum circuits. Another approach is to use a control-pulse to engineer the quantum gate. It can be used to implement a high-fidelity and fast quantum gate with a control-pulse sequence instead of a deep quantum circuit through optimal gate synthesis, thereby reducing the computational overhead. This approach utilizes the use of unrestricted unitary gates contrary to standard universal gate set approach, thereby decreasing the required circuit depth by one-order of magnitude.[17]

The major notable issues with the control sequence approach is the accountability for various error/noise corrections that needs to be made during experiments. The restriction to induce any desired quantum evolution owing to a specific form of the system Hamiltonian which is limited by underlying physics of the quantum system on hand. The complexity increases exponentially with the number of qubits involved. The structure of high-dimensional control landscapes of multi-qubit systems and errors corresponding to them are poorly understood owing to lack of tools and high computational cost. However, unsupervised machine learning techniques are able to obtain precise and scalable solutions.

One of the well known machine learning solutions for solving sequence to sequence related problems are witnessed in the domain of natural language processing (NLP). The foundations required

to understand machine learning and the methods used to solve known NLP tasks are discussed in chapter 3. Remarkably, Reinforcement learning (RL) performs the best, on unlabeled data and is resistant to sample noise owing to its effectiveness and stability towards stochastic nature of the environment. In RL, an intelligent agent takes sequential actions trying to maximize a reward function that drives us to the optimal solution to the target in the problem. The training of the agent depends on balancing exploration of the unknown territory of the given control landscape to gain knowledge, with exploitation of the gained knowledge. More details about the methods will be discussed in chapter 5.

1.2 Previous Work

The optimal control is performed in general using gradient descent approaches using GRAPE, krotov and other SGD approaches like Nealer-Mead, Powell etc., the performance decreases drastically with increasing dimensionality of control space owing to its inability to get out of local traps. Simulated annealing, genetic algorithms like differential evolution, approaches can be used to avoid local traps.

Barry Sanders et. al, [6] used differential evolution to implement Subspace Selective Self-Adaptive Differential Evolution (SuSSADE) on a system comprising three nearest neighbor-coupled superconducting artificial atoms with parameters appropriate for the transmon system to achieve fidelity of close to 0.9999. This is shown to be producing similar results for Fredkin, CXX, CCZ and CZZ gates.

Marin Bukov et. al, [15] exploited Q-learning for single-qubit and a chain of coupled qubit systems obtaining 0.9999 fidelity discretizing the range of pulse to obtain optimal pulse sequence. They explored the computational space for the first 40 episodes and then utilized experience replay to exploit the knowledge gained during the training phase.

Mahesh et. al, [21] in the paper on push-pull optimization of quantum controls had explained dividing operators along the target operator and orthogonal to the target operator improved the precision over general gradient based and variational principles. The idea is to minimize the values along orthogonal operators (push away) and maximize the values of operators along the target operator.

David Schuster et. al [23] implemented GPU-based GRAPE method to prepare high fidelity gates. Elija Perrier et. al [7] used differential geometry approach to verify time optimal routes and implement those gates. Zheng An et. al [1] implemented dueling double deep Q-learning neural network to find optimal control sequence to implement a single-qubit Hadamard gate and a two-qubit CNOT gate. They used bang-bang protocol for defining the control sequence. The stability of the model provides insight into using deep neural network to quickly learn the given quantum control landscape.

We utilize the quantum system dynamics of three-qubit transmon system mentioned in [6], and utilize sequence to sequence models, deep RL techniques namely Actor-Critic Network architecture and apply Trust Region Policy Optimization (TRPO) using continuous range protocol for defining control sequences to find optimal pulse sequence for CCZ gate. The implementation details are mentioned in chapter 6. We also implement trust region policy optimization on single-qubit and two-qubit gates, using continuous range and bang-bang protocol for the control sequences, and implementation details are discussed at greater detail in chapter 5 and 6. We then compare the results obtained with existing baselines in chapter 7 and present future work in chapter 8.

Chapter 2

Quantum Computing

2.1 Quantum Computing

Quantum computing is a sub field of quantum information science and uses the quantum phenomena like superposition and entanglement to perform computation and gain knowledge from the information. In this section, we give a brief introduction to quantum computing by presenting results taken from external sources, without proof, to give the reader an eagle's eye view.

2.1.1 Bits and Qubits

In classical computers, the bit is the fundamental mathematical object of the computation and information. Every classical bit has a state, which is either 0 or 1. Most of the modern computers are built of this two-state system. Precisely, the classical circuits are built upon the classical bits determining the computation. But, for all purposes, any system with a finite set of discrete and stable states with controlled transitions between them can be used to build computers.

There exists a similar fundamental mathematical object for quantum computers, a qubit (short form for Quantum Bit). The difference between qubit and classical bit is the former can exist in superposition of states.

The general state of a qubit can be described as $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, Where α, β are complex numbers and $|\alpha|^2 + |\beta|^2 = 1$. Every such state can be represented as

$$|\psi\rangle = e^{i\gamma}(\cos(\frac{\theta}{2})|0\rangle + e^{i\phi}\sin(\frac{\theta}{2})|1\rangle)$$

Let's understand few mathematical concepts that are required. Bloch Sphere can be used as a geometric representation of the pure state of a two-level quantum system (qubit).

The state space of any physical system is the complex inner product space which is also known as Hilbert Space. The system, at any given time can be described using the state vector, which is a unit vector in the state space. The quantum state of a qubit $|\phi\rangle$ is a vector in $\mathbb{C}^2$, two dimensional complex vector space. This can be generalized for a n-qubit system as tensor product of $\mathbb{C}^2$ n-times leading to $\mathbb{C}^{2^n}$.

Any vector v in the vector space $\mathcal{V}$ can be represented as linear combination of minimal collection of n vectors (i.e, the dimensionality of the $\mathcal{V}$ vectors) $|v_1\rangle, |v_2\rangle \dots |v_n\rangle$ called as basis vectors. In simpler terms, any vector v can be represented as $\dim(\mathcal{V})$ -tuple of scalars. The standard basis for $\mathbb{C}^n$ would be $[1, 0, \dots, 0]^T, [0, 1, 0, \dots]^T, [0, \dots, 0, 1]^T$

A linear operator $\mathcal{A}$ is defined from a vector space $\mathcal{V}$ to $\mathcal{W}$ such that,

$$\mathcal{A}(\alpha u + \beta v) = \alpha \mathcal{A}u + \beta \mathcal{A}v$$

given the dimension of $\mathcal{V}$ and $\mathcal{W}$ as m and n respectively, the operator $\mathcal{A}$ can be represented as $m \times n$ matrix.

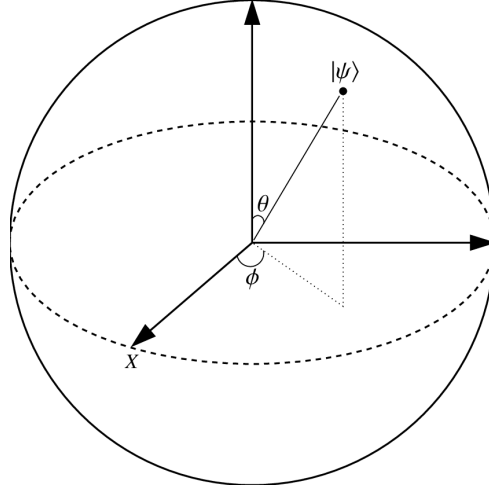


FIGURE 2.1: Representing quantum state $|\psi\rangle$ using bloch sphere

The bra-ket notation, $|\cdot\rangle$ represents vector in a hermitian vector space and $\langle\cdot|$ is its complex conjugate transpose. The inner product of $|u\rangle = \sum_{i=1}^n u_i |i\rangle$, $|v\rangle = \sum_{i=1}^n v_i |i\rangle$ is

$$\langle u|v\rangle = \sum_{i=1}^n u_i^* v_i$$

and the outer product is denoted as $|u\rangle\langle v|$. The outer product with itself $|u\rangle\langle u|$ is the projection on the one-dimensional space generated by $|u\rangle$. Also, any linear operator can be expressed as linear combination of outer products, i.e,

$$\mathcal{A} = \sum_{ij} \mathcal{A}_{ij} |i\rangle\langle j|$$

2.1.2 Quantum Mechanics

Quantum Mechanics is a general framework that describes the characteristics of the physical systems and helps in development of physical theories. The key ideas helps us define the state space of the system, behaviour of the system with time and its interactions with the external physical systems. Following are the key ideas,

- Each physical system is associated with a topologically separable complex Hilbert space $\mathcal{H}$ also known as state space, i.e, the system is completely described by its state vector, a unit vector in the state space
- The time evolution of the closed quantum system is described by a unitary transformation and is given by Schrodinger equation,

$$i\hbar \frac{d|\phi\rangle}{dt} = \mathcal{H}|\phi\rangle \quad (2.1)$$

where $\mathcal{H}$ is Hamiltonian of the system.

- The measurement operators $\mathcal{M}_m$ satisfy the completeness equation,

$$\sum_m \mathcal{M}_m^\dagger \mathcal{M}_m = \mathcal{I} \quad (2.2)$$

From 2.1,

$$\frac{d|\phi\rangle}{|\phi\rangle} = \frac{-i\mathcal{H}dt}{\hbar} \implies d(\ln(|\phi(t)\rangle)) = \frac{-i\mathcal{H}dt}{\hbar} \quad (2.3)$$

$$\ln\left(\frac{|\phi(t)\rangle}{|\phi(0)\rangle}\right) = \int_0^t \frac{-i\mathcal{H}dt}{\hbar} \quad (2.4)$$

$$|\phi(t)\rangle = e^{\int_0^t \frac{-i\mathcal{H}dt}{\hbar}} |\phi(0)\rangle \quad (2.5)$$

The Hamiltonian is a hermitian operator, which implies that $e^{\frac{-i\mathcal{H}dt}{\hbar}}$ would be a unitary matrix $\mathcal{U}$, so, $|\phi(t)\rangle = \mathcal{U} |\phi(0)\rangle$

2.2 Classical Gates and Quantum Gates

In classical computers, the computations are performed by transitioning a set of bits into another using classical gates. A classical computation can be stated as a function that takes n-bit input and returns a m-bit output which can be determined uniquely depending on input, in other terms, function $f: \{0,1\}^n \rightarrow \{0,1\}^m$. For example, AND gate takes input of two classical bits a, b and output a value c which is in-turn a classical bit. From Landauer's principle, the erasure of the information is a dissipative process. The classical computation done in the above example is irreversible, i.e, one or more pairs of kind (a,b) can result in output c and there's no way to figure out (a,b) pair without doing external work by spending energy.

In classical computation, every gate can be decomposed using a universal gate set, which is set of $\mathcal{U} = \{\text{NOR}\}$ or $\mathcal{U} = \{\text{NAND}\}$ gates or $\mathcal{U} = \{\text{AND}, \text{NOT}, \text{OR}\}$. That implies, every complex gate can be converted into finite decomposition of one of gates in $\mathcal{U}$.

The Quantum gates on the other hand are operated on n-qubits resulting n-qubits and given the output, we can exactly determine what is the input. Every quantum gate can be mathematically represented as a unitary operator, U_i . There exists universal gate set for quantum gates.

2.3 Quantum Gates and their Decomposition

Theorem 1 Any unitary transformation of any number of qubits n can be decomposed as a product of 1- and 2-qubit gates

Theorem 2 "Almost all" unitaries require exponentially large quantum circuits of 1- and 2- qubit gates to implement them exactly.

Definition 1 (Universal Quantum Gate set) A set S is called universal if by composing the gates in S , any unitary transformation on any number of qubits can be approximated to any desired degree of precision. The set is closed under inverses if $\forall$ gates $s \in S$, $s^{-1} \in S$.

Theorem 3 Solovay-Kitaev Theorem - if a gate set S is closed under inverses and is universal then any unitary on n qubits can be approximated to within ϵ precision using $O(4^n \text{polylog}(\frac{1}{\epsilon}))$ gates from the set S .

The Solovay-Kitaev Theorem says that if we can implement the finite number of gates in a universal gate set closed under inverses, we can implement any unitary to arbitrary precision. However, the number of gates suggested by the theorem is practically impossible for even as little as three qubits where for a precision of 10^{-3} , close to 200 gates are required. Coherence time for superconducting qubits is around $50\mu\text{s}$. The typical gate times are around 130ns for single qubit and 250 - 450ns [linke] for two qubit gates. A back of the envelope calculation would reveal that the time scales for implementing a three-qubit gate are comparable to the coherence time thus resulting in bad gate fidelities due to large decoherence, which presents the need to come up with a better way to implement quantum gates.

One should remember that the Solovay-Kitaev Theorem builds gates using an algorithmic approach which may not necessarily be the optimal solution with the given gate set. There do exist results of optimality such as those by Shende and Markov [26] which prove that a Toffoli gate requires 6 CNOT and 6 single qubit gates at least to implement it.

2.4 Quantum Control

Given that synthesis of quantum gates from gate set will become exponentially hard as we increase the qubits in the system. We look for better ways to implement the quantum gates using control pulses.

In optimal control theory, Pontryagin's maximum principle is used to find the best possible control for taking a dynamical system from the initial state to a target state, in the presence of constraints for the state or input controls. The principle states that it is necessary for any optimal control along with the optimal state trajectory to solve the Hamiltonian system, which is a two-point boundary value problem, and a maximum condition of the Hamiltonian. The significance of the maximum principle lies in the fact that maximizing the Hamiltonian is much easier than the original infinite-dimensional control problem. Equivalently, the problem is converted to a pointwise optimization, rather than maximizing over a function space. Similar logic leads to Bellman's principle of optimality, used in reinforcement learning, is a related approach to optimal control problems which states that the optimal trajectory remains optimal at intermediate points in time.

2.4.1 Forward and Inverse Hamiltonian Problem

Forward Hamiltonian problem can be defined as given an initial state $|\phi(0)\rangle$ and Hamiltonian $\mathcal{H}(\epsilon(t))$ of the system and the variation of $\epsilon(t)$ with time, we need to solve for the state of the system at time t , $|\phi(t)\rangle$.

From the equation 2.5, the solution can be obtained easily. We will just integrate the function, $\frac{-i\mathcal{H}(\epsilon(t))dt}{\hbar}$ from $t = 0$ to $t = t$ to find the state of the system at time t .

Let's think about the inverse problem, given the initial state of the system $|\phi(0)\rangle$ and Hamiltonian $\mathcal{H}(\epsilon(t))$ of the system the state of the system at time t , $|\phi(t)\rangle$, we need to determine the variation of $\epsilon(t)$ with time. This is a challenging optimization problem, which requires us to search the entire Hilbert Space $\mathcal{H}$.

It can be put more formally as, given the initial state $\phi(0)$, final state $\phi(t_n)$ the hamiltonian $\mathcal{H}(\epsilon(t_i))$, say U_i is the unitary obtained by applying pulse of $\epsilon(t_i)$ during the i^{th} timestep then, find the sequence $\epsilon(t_0), \epsilon(t_1), \dots, \epsilon(t_n)$ such that,

$$\phi'(t_n) = U_n U_{n-1} \dots U_2 U_1 U_0 \phi(0), \quad (2.6)$$

the fidelity of $\phi'(t_n)$ with respect to $\phi(t_n)$ is as close to 1 as possible.

2.4.2 From Control pulses to Unitaries

Transmon system

In physical systems control is at the level of some physical parameter such as voltage or flux. We thus have a continuous system providing us with an uncountably many number of unitaries unlike the case of Gate decomposition where we had finitely many gates in our toolbox. This could potentially allow us to implement arbitrary gates with high speed and fidelity. In the paper by Barry Sanders et al. (2016)[6] a Hamiltonian with three time dependent parameters is used. The Unitary is calculated by performing a time ordering integral and using fidelity as the fitness function. The Hamiltonian is the following:

$$\frac{\hat{\mathcal{H}}(t)}{\hbar} = \sum_{k=1}^3 \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & \epsilon_k(t) & 0 & 0 \\ 0 & 0 & 2\epsilon_k(t) - \eta & 0 \\ 0 & 0 & 0 & 3\epsilon_k(t) - \eta' \end{pmatrix}_k + \frac{g}{2} \sum_{k=1}^2 (\hat{X}_k \hat{X}_{k+1} + \hat{Y}_k \hat{Y}_{k+1}) \quad (2.7)$$

where $k = 1, 2, 3$ is the transmon index. ϵ is the control parameter, $\eta' = 3\eta$ are the anharmonicities of the second and third level. Transmons are capacitively coupled with strength g . Experimentally, ϵ

lies between -2.5 and 2.5 GHz, $\eta = 200$ MHz and $g = 30$ MHz. The problem is to determine a pulse sequence ϵ such that the unitary implemented is our target unitary. To simplify the problem the pulse sequence is completely defined by a finite number of control parameters (values of the pulse at some discrete points) which are interpolated by an error function that emulates what actual electronics would implement. The Hamiltonian is evolved to get the unitary using

$$U(\Theta) = \hat{T} \exp \left(\int_0^\Theta -i\hat{H}_p(\tau) d\tau \right) \quad (2.8)$$

where $\hat{H}_p$ is defined as

$$\hat{H}_p(t) = \hat{O}_3 \hat{H}(t) \hat{O}_3' \quad (2.9)$$

where $\hat{O}_3$ is the projection operator onto the 20 state subspace of energy levels with at most 3 excitations.

In our results we use a 30ns second pulse sequence where a control parameter is placed at every nanosecond. The fitness function used to determine whether the child or the parent passes on to the next generation is

$$\mathcal{F} = \frac{1}{8} \left| \text{Tr}[U'_{target} U_{cb}(\Theta)] \right| \quad (2.10)$$

Lastly, since single qubit rotations are regarded as trivial, an initial and final phase is subtracted in such a manner that it maximises the fidelity. More formally $U_{pre,post}$ defined as

$$U_{pre,post} = \text{diag}(1, e^{-i\beta_3}, e^{-i\beta_2}, e^{-i(\beta_2+\beta_3)}, e^{-i\beta_1} e^{-i(\beta_1+\beta_3)}, e^{-i(\beta_1+\beta_2)}, e^{-i(\beta_1+\beta_2+\beta_3)}) \quad (2.11)$$

and the angles in the pre and post matrices are chosen such that

$$\mathcal{F}(U_{pre}(\alpha_1, \alpha_2, \alpha_3) U_{cb}(\Theta) U_{post}(\beta_1, \beta_2, \beta_3)) \text{ is maximised.} \quad (2.12)$$

Chapter 3

Machine Learning

3.1 Formalism

Machine learning is the study of computer algorithms that improves automatically from experience and the knowledge gained through data provided. This paradigm can be used to solve problems related to optimization, mathematical modelling, prediction modelling, classification, malware detection, natural language processing, image processing, computer vision and many more.

The core idea behind most of the machine learning algorithms is to understand the system using the data by learning key features present in them through minimization of loss function, which, in most cases would be an indicator of discrepancies between the given data and the predictions made on the data. The prediction process can be seen as a pattern recognition in the data and using the knowledge of these patterns, the machine can predict relevant information about unseen data. It is called learning from experience because, whenever the prediction is deviated significantly from the expectation, correction is made during the training to the system to minimize these deviations, thereby improving its ability to predict more precisely. There are three main approaches using this paradigm based on nature of input data, namely, supervised learning, unsupervised learning and reinforcement learning.

3.1.1 Supervised learning

It is a machine learning task, where the machine should predict the mapping from input to output based on the labelling in training data. Handwriting analysis, support vector machines, decision trees etc., come under this category.

3.1.2 Un-Supervised learning

This corresponds to the machine learning task, where the training data is not labelled but the information about the data is to be build using some compact internal representation using cluster analysis or relevant metrics.

3.1.3 Reinforcement learning

This corresponds to the machine learning task, where the training data is abundant and is obtained by interacting with the environment which can be used to build compact internal representation using MDP(markov decision process) and other relevant metrics. More about this is discussed in next chapter.

3.1.4 Learning workflow

The typical workflow of any machine learning task would start with data collection. Then, this data should be prepared to implement the necessary machine learning algorithm accordingly. Data preparation includes, data cleaning and pre-processing. In data cleaning, the outliers and corrupted values are removed. In pre-processing, unnecessary data is removed and the relevant is modified from initial form to desired form. Now, we have the data in the required form to implement the desired machine learning algorithm.

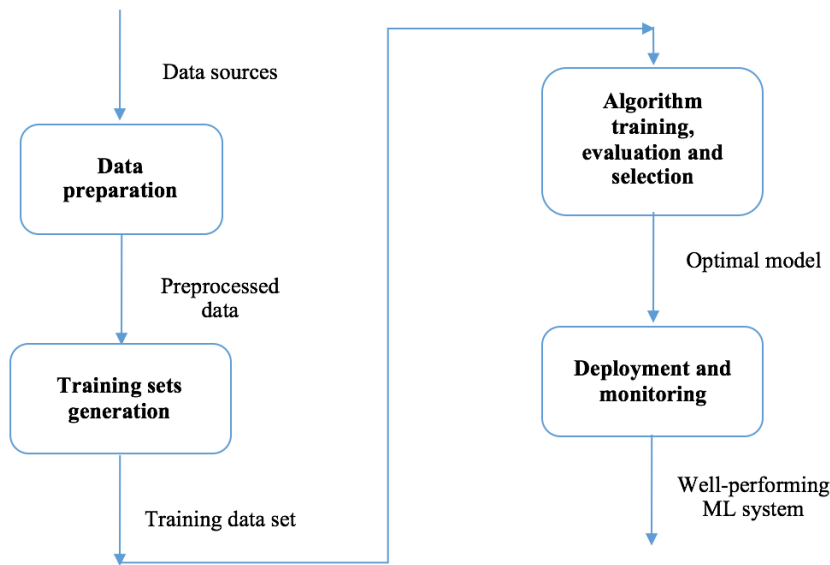


FIGURE 3.1: ML workflow

Machine learning algorithm differs from task to task, at-times, we might need to do feature engineering by analyzing the data and figuring out the relevant features and sometimes it left to deep neural networks to model the features for itself. But, loss function or objective function should be designed accordingly to the task so that it can learn useful information from the data provided. If, neural networks are used, the design of the network architecture and tuning the relevant parameters (weights of the network) and hyper-parameters will become crucial. Feedback is provided to the system through the objective function, so as to make corrections to the parameters to learn the task better. This is the core idea behind training the agent to perform the provided task.

After training process, we use the learned agent to measure the performance. If the performance does not meet the desired standards, the process is repeated after making necessary tweaks. This is an iterative process and is continued till the desired standard is met.

3.2 Artificial Neural Networks

Artificial neural networks (ANNs) are the connectionist computing systems that are inspired from the biological network of neurons that constitute animal brains. Such systems have the ability to progressively improve to do the tasks by considering examples, generally without any prior task-specific programming. They are able to learn to differentiate between unseen images of birds and animals when provided with abundant examples of labelled images of birds and animals prior to it, by analyzing these examples analytically. They have found most use in applications that are hard to express with rule-based programming in traditional computer algorithms.

The basic unit of ANN is called artificial neuron, and ANN is a collection of artificial neurons connected in a specific manner. Similar to biological neurons in the biological brain, each connection in the network can transmit a signal to another neuron. The signal is transmitted to the receiving neuron is then processed and then signal downstream neurons connected to it. To restrict the activity of neurons based on the value it holds, we use activation function to pass on the information. Typically, neurons are organized in layers. Different layers may perform different kinds of transformations on their inputs. Signals travel from the input layer, to the last output layer, possibly after traversing the layers multiple times. The goal of the neural network is to learn to solve problems in the way a human brain would. Over time, attention focused on matching specific mental abilities, leading to deviations from biology such as back-propagation, or passing information in the reverse direction and adjusting the network to reflect that information.

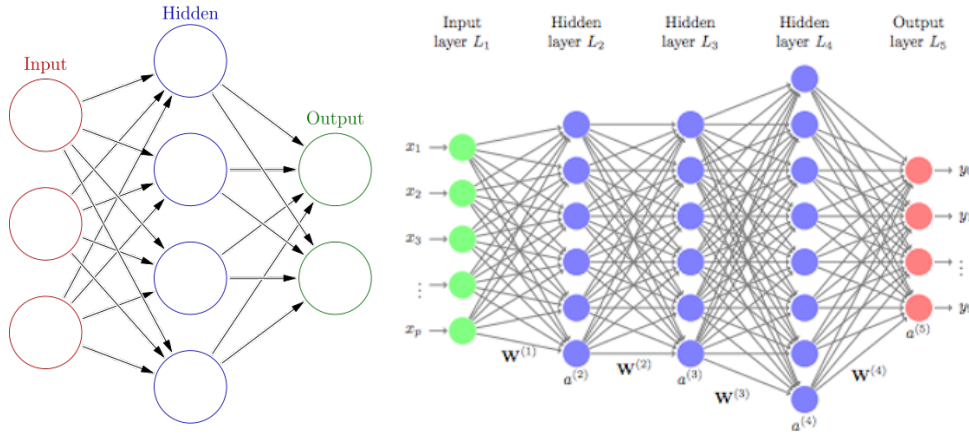


FIGURE 3.2: Artificial Neural networks and Deep Neural Networks

The generally the first layer of the neural network is called the input layer and the last layer is called the output layer and the layers in-between are called hidden layers. If there are multiple layers in-between the input and output layer the ANN is called Deep Neural Network(DNN).

3.3 Different types of networks

Neural networks have been used to solve variety of tasks, including computer vision, speech recognition, machine translation, playing board games, playing video games, and medical diagnosis. Each of these solutions include differently connected set of neurons and at times the architecture is tailored specific to the needs. Also, learning a task includes using appropriate loss functions to change the weights of the network, in-order to correct its mistake through backpropagation. The following are different kinds of neural networks that are generally used.

3.3.1 Feed-Forward Neural Network

Feed-forward neural networks are the neural networks with connections not forming any cycle or loop. Well known examples are single-layer perceptron, multi-layer perceptron and radial basis network. In feed-forward network, the hidden layers have their inputs and outputs masked by the activation function. The connections between layers can be fully connected or partially connected. Specific way of connections are found useful for specific problems.

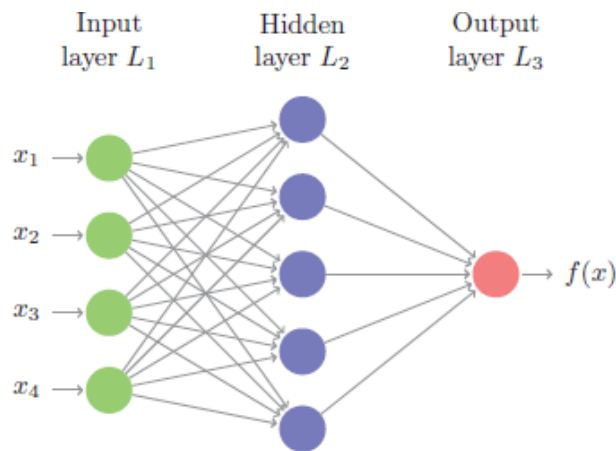


FIGURE 3.3: Feed Forward Neural Network

3.3.2 Convolutional Neural Network

Convolutional neural networks (CNNs) consists of input layer, output layer and hidden layers that perform convolutions, where as in a feed-forward network, the hidden layers have their inputs and outputs masked by the activation function and final convolution. Typically this convolution layer includes a layer that performs a dot product of the convolution kernel with the layer's input matrix. This product is usually the Frobenius inner product, and its activation function is commonly ReLU. As the convolution kernel slides along the input matrix for the layer, the convolution operation generates a feature map, which in turn contributes to the input of the next layer. This is followed by other layers such as pooling layers, fully connected layers, and normalization layers. This kind of neural network is used mostly when images are involved, where the nearby pixels together give us information about features like boundaries, edges, color shift.

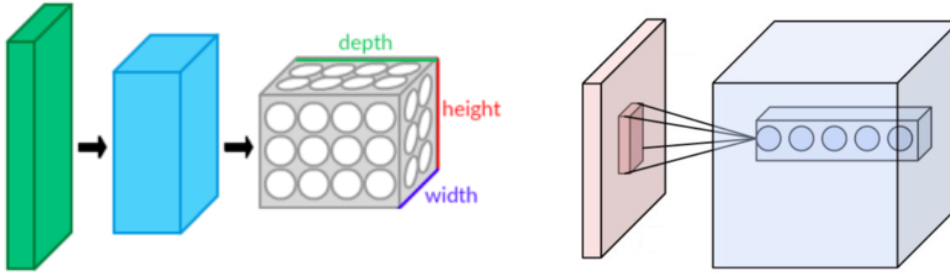


FIGURE 3.4: CNN unit 3-D representation and mapping of units between layers

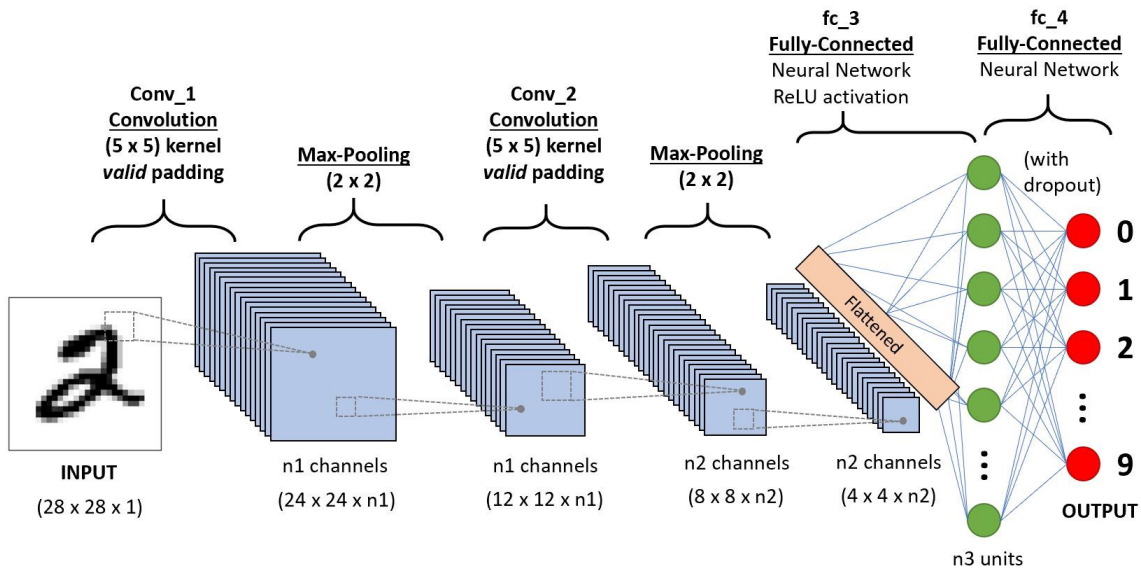


FIGURE 3.5: Convolutional Neural Network using digit recognition example

3.3.3 Recurrent Neural Network

Recurrent neural network (RNN) is a class of artificial neural networks (ANNs) which has a directed graph like connections between nodes along a temporal sequence. Derived from feed-forward neural network, this additional property allows to exhibit temporal dynamic behaviour. These are used for the problems involving temporal data. RNNs can use their internal state, also called memory, to process variable length input sequences, making them useful for the tasks like unsegmented but connected handwriting recognition, natural language processing and speech recognition.

There are two broad classes of networks with similar general structure under the umbrella of the term, recurrent neural network. One is with finite impulse and another is with infinite impulse. The

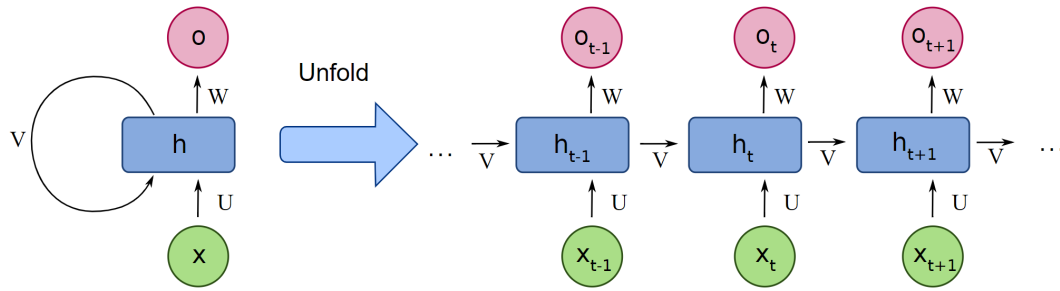


FIGURE 3.6: Recurrent Neural Network

key distinction is that a finite impulse recurrent neural network is a directed acyclic graph that can be unrolled and replaced with a feed-forward neural network, while an infinite impulse recurrent network is a directed cyclic graph that cannot be unrolled.

The temporal data is efficiently handled by both of these networks, as they can have additional stored states and the storage can be under direct control of the neural network. The storage can be replaced by another network or graph, if there is any need to incorporate time delays or feedback loops. Such controlled states are referred to as gated state or gated memory. These are a part of well-known long short-term memory networks (LSTMs) and gated recurrent units (GRUs). These are also called as feedback neural networks.

3.3.4 Auto-Encoders

Autoencoder is a type of Artificial Neural Network used to learn patterns in the data by encoding it in an unsupervised manner. The core idea is to represent the underlying pattern in the data, by training the network to maintain a hidden representation and use this representation to learn about useful features of the data, typically by dimensionality reduction. Along with the reduction side, a reconstructing side is learned, where the autoencoder tries to generate from the reduced hidden encoding representation as close as possible to its original input. There by trying to reduce unnecessary noise in the input. There are different types of autoencoders used for different purposes, regularized autoencoders are effective in learning representations for subsequent classification tasks, variational autoencoders has applications as generative models. Autoencoders are also applied to many problems, from facial recognition to acquiring the semantic meaning of words.

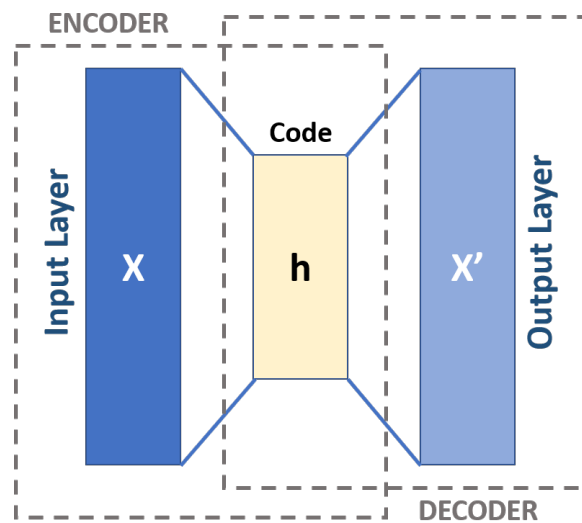


FIGURE 3.7: Auto-encoder

The other neural networks include, Hopfield networks, Helmholtz networks and Boltzmann Machines as shown below.

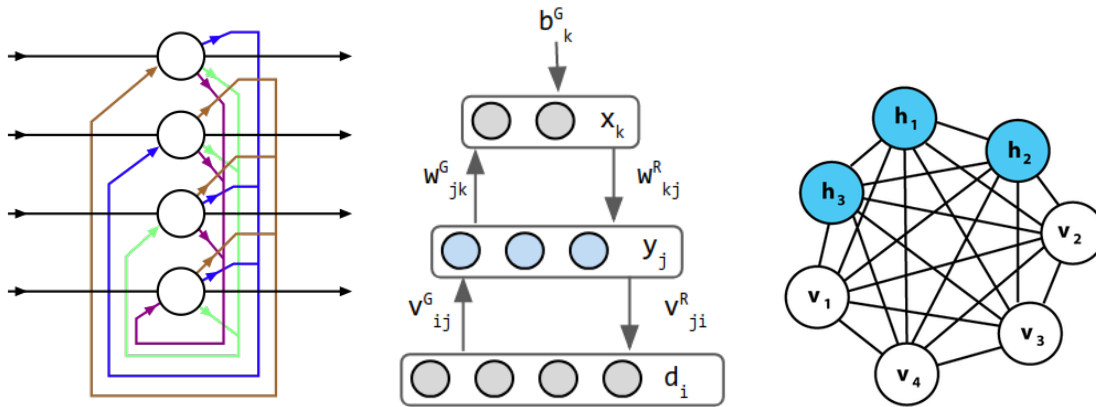


FIGURE 3.8: Hopfield, Helmholtz networks and Boltzmann machine

3.4 Natural Language Processing

Natural Language Processing (NLP) deals with the ability to understand and process the sentences from a given language, to perform tasks accordingly or to translate from given language to another. Any language is built over a set of rules, structural constraints on the composition of phrases, clauses and words, also known as grammar of the language. The users of the language should abide by these set of rules. Even, the machine trying to learn the language should be able to learn the patterns and set of rules, out of which some are dependent on the relative context. Some of these rules are ambiguous and the machine or the learner should be able to respond without ambiguity.

The process is to understand the underlying rules by encoding it to mimic the core representation of the language, which requires understanding of the context and temporal data. The natural language processing research relies on machine learning techniques. One of the most used techniques is to automatically learn such rules through the analysis of large corpora of the language using statistical inference. The corpora is instances of real-world examples from set of documents or human or computer annotations.

Different classes of machine learning algorithms have been used to perform natural language processing tasks. The input for these algorithms would be a large set of features extracted from the input data. There have been innovative methods used to extract relevant features that are useful for these algorithms. Increasingly, the research in this domain has focused on statistical models, which make probabilistic decisions based on attributing real valued weights to each input feature. These models have advantage of expressing different possible answers with relative certainty that are more reliable than providing just only one answer when they are included as a component of a larger system.

While mimicing the rules of the language, we can observe, the next word of the sentence depends on the latest used words in the sentence and also should follow a set of rules similar to if-then rules that are used in programming. The earliest-used machine learning algorithms, such as, decision trees, produced systems that are based on strictly restricted if-then rules similar to the existing hand-written rules. Each individual word doesn't make sense without the context, however, if the sentence is tagged using part-of-speech, then each word has information about the context. Part-of-speech tagging, introduced the used of hidden Markov models to natural language processing. These can be further used to engineer the relevant features. The research on focused on statistical models, makes use of probabilistic decisions attaching real valued weight to the engineered-features.

The cache language models upon which many speech recognition systems now rely are examples of such statistical models. These models are more robust when unseen input data is provided, especially input that contains errors, and produce more reliable results when integrated as subsystem of larger systems.

The cache models might not be scalable if elaborate feature engineering is required. The neural networks comes for the rescue here. Most of the statistical methods are replaced by neural networks. The main idea include the use of word-embeddings to capture the semantic properties of words, it is a way to express the word as a matrix of real valued number after observing the context of the given word in various different contexts. This technique has increased, end-to-end learning of higher level tasks like question answering without relying on a pipeline of individual intermediate tasks like part-of-speech tagging and dependency parsing.

The shift from statistical methods to deep neural networks can be viewed as paradigm shift to emphasize more on sequence-to-sequence learning. Given a sequence of words as input, the deep networks are expected to figure out the relevant features from the word-embeddings of the input data after pre-processing to produce the desired sequence of words as output. The pre-processing part includes removing the stop-words, the words that might not be useful for interpreting the sentence. Replacing the parts-of-speech tagging with word-embeddings provides us with the flexibility to represent various problems like machine translation, question-answering, text summarization as sequence to sequence learning problems.

3.5 Sequence to sequence models

Sequence to sequence (Seq2seq) models constitute the framework for solving for input sequence of certain data units and the output of sequence of not necessarily similar data units which is common pattern for sequential problems. These models are mostly trained using a ground truth sequences. seq2seq models are used in various applications including machine translation, question-answering, text summarization, speech-to-text applications and image captioning.

The general framework for solving problems based on sequence to sequence learning is similar to encoder-decoder architectures. Using deep neural networks, we can solve using various methods like, encoder-decoder with beam search, encoder-decoder with attention models, and encoder-decoder by using RNNs like long short-term memory (lstm).

To understand the use of encoder-decoder architecture is crucial, because, the input sequence might not be in the same data units as the output and the underlying representation of translation from input to output, requires a mixed coded representation and a decoder to translate this coded representation back to data units of the output sequence. The sequential models would be dealing with temporal data, where the use of long short-term memory units arises, because the knowledge of the recent events helps to represent the current state better than when we don't have it.

There are also bi-directional long short-term memory units (BiLSTM) which in contrast to LSTMs, passes the information in both directions to improve the context available for the algorithm. The core idea is to effectively use the existing neural networks and combine them into an architecture which can increase the information about the context that can be provided to the algorithm to improve the performance of the seq2seq models.

BERT, Bidirection Encoder Representations from Transformer, is one of the most famous language model used for NLP tasks and is made by google. To understand the complexity of the problem, BERT (large) model is an architecture made of 24 encoders and 16 self-attention heads, and is pre-trained on data from BooksCorpus consists of 800 million words and English Wikipedia with 2500 million words.

3.6 Control problem as seq2seq problem

There are certain similarities between the quantum control problem we are trying to solve and the known sequence to sequence problem from natural language processing domain. The most important similarity is sequential nature of the problem, time-dependent varying nature based upon certain set of underlying rules, Hamiltonian system and time evolution for the former and language grammar for the later. Also, the length of the sequence is not fixed in both the cases, we can get best fidelity for two sequence of different length and attaining best fidelity for smaller sequences and for larger sequences

are difficult, similar to the way making meaningful single sentence with way lesser words or with larger number of words is difficult.

The underlying control landscape for the control problem is fixed and depends on the physical system we can try to control, the perturbations caused because of noise will not change the control landscape, but the control sequence we try to implement and we need to find that sequence which is robust to the type of noise that is interfering with the system. Similarly for the sequence problems, the rules are well defined and there might be multiple reliable solutions, which are devised using probabilistic models. So, the control problem is similar to sequence-to-sequence learning problem, where the initial sequence is a random pulse sequence and output sequence is the optimal pulse sequence which produces best fidelity for the given quantum gate.

Chapter 4

Reinforcement Learning

4.1 Formalism

Reinforcement learning[28] is the problem faced by an agent that learns behavior through trial-and-error interactions with a dynamic environment. It mainly models the problem into a Markov Decision Process (MDP). The key feature of reinforcement learning is that it explicitly considers the whole problem of a goal-directed agent interacting with an uncertain environment. This is in contrast with many approaches that consider sub-problems without addressing how they might fit into a larger picture.

Definition 2 (Markov Property) *A stochastic process has the Markov property, if the conditional probability distribution of future states of the process (conditional on both past and present states) depends only upon the present state, not on the sequence of events that preceded it*

Markov decision processes (MDPs) are problems of sequential decision-making in which a control (action) has to be selected in each decision-making state visited by the concerned system. In an MDP, the system under consideration is assumed to be driven by underlying Markov chains. In each such Markov chain, the system jumps randomly from one state to another in discrete time steps, and the probability of transition from the current state to the next depends only on the current state and not on where the system has been before. That is the state transitions in the MDPs satisfy Markov property.

A policy defines the action to be chosen in every state; it is a mapping from the set of states to the set of actions. An immediate reward, which may be positive, negative or zero, is earned in transitioning from one state to another under the influence of an action.

Each MDP consists of five important elements,

- State Space (S)
- Action Space (A)
- Transition function (T)
- Reward function (R)
- Discount factor (γ)

State space consists of set of all encoding of possible states of the environment on which we are implementing our control policies. Action space consists of all possible actions that can be taken from any random state in state space to reach another state. Given a state s , another state s' and action a , $T(s,a,s')$ provides the probability with which that transition from s to s' under action, a , occurs. Reward function gives the value that are associated with each transition 3-tuple (s,a,s') which we want to maximize by the time we reach the goal state. Discount factor determines the importance of later actions.

Given a policy π , which determines the action that needs to be taken at a given state s , then value function for such policy (π) v_π is given by,

$$v_\pi(s) = E(R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s)$$

Similarly Q-value of policy (π) Q_π is given by,

$$Q_\pi(s, a) = E(R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s, A_t = a)$$

Optimal policy is that policy which results in maximum cumulative reward. By choosing that action a' which results in $a' = \underset{a}{\operatorname{argmax}}(Q_\pi(s, a))$, we ensure to get maximum cumulative reward.

4.2 Q-learning

Q-learning is an off policy reinforcement learning algorithm that seeks to find the best action to take given the current state. It is considered off-policy because the Q-learning function learns from actions that are outside the current policy, like taking random actions, and therefore a policy is not needed. More specifically, Q-learning seeks to learn a policy that maximizes the total reward.

Q-learning works by incrementally updating the expected values of actions in states. For every possible state, every possible action is assigned a value which is a function of both the immediate reward for taking that action and the expected reward in the future based on the new state that is the result of taking that action. This is expressed by the one-step Q-update equation,

$$Q(s, a) := (1 - \alpha)Q(s, a) + \alpha(R_t + \gamma * (\max_a Q(s_{t+1}, a)))$$

where Q is the expected value of performing action a in state s ; s is the state vector; a is the action vector; R_t is the reward; α is a learning rate which controls convergence and γ is the discount factor. The discount factor makes rewards earned earlier more valuable than those received later.

In our problem, we have three values, $\epsilon_1, \epsilon_2, \epsilon_3$ to determine the state at any time t . So, the state space for this problem can be encoded as,

$$\mathcal{S} = (t, \epsilon_1, \epsilon_2, \epsilon_3)$$

The action space will be a 3-tuple which measures change in each of ϵ_i given by $\delta\epsilon_i \forall i$ in $\{1, 2, 3\}$. We try to find the sequence till the target time $T = 30\text{ns}$, each iteration is of $\delta t = 1 \text{ ns}$.

$$R_t = \{0 \text{ if } t < T; \text{ fidelity}, \mathcal{F} \text{ if } t = T\}$$

The main problem with this method is to explore the infinitely large state space is not computationally possible. Even for the enumeration of discretized state space into $n = 100$ buckets for each integer value ϵ_i is computationally impossible. As the memory required to store the Q-values would be

$$\frac{T}{\delta t} n^{\{no.of\ actions + no.of\ states\}} = 30 * 100^6 = 3 * 10^{13} = 30000\text{GB}$$

We then tried discretizing the state space into 10 buckets for the window of ϵ_i values to be in the range $[-3, 3]$ giving us the quantum change of value to be 0.6. We then verified whether this method gives us the best possible fidelity for these values. It does give us the best possible value for the set of states. But it performs poorly than DE because of lack of capacity to learn information about the environment which is a key mechanism for RL learning methods.

For handling this unavailability of compute, we need to try Policy gradient, Deep RL methods which approximates the Q-value of each state, action pair into a function using parameterized evaluator and linear function approximator methods.

4.3 Policy Gradient methods

The goal of reinforcement learning is to find an optimal behavior strategy for the agent to obtain optimal rewards. The policy gradient methods target at modeling and optimizing the policy directly. The policy is usually modeled with a parameterized function respect to θ , $\pi_\theta(a|s)$. The value of the reward (objective) function depends on this policy and then various algorithms can be applied to optimize θ for the best reward. Policy gradient methods frame the goal of maximizing return as the minimization of a loss function $L(\theta)$ where θ parameterizes the agent.

4.3.1 ACTOR-CRITIC

Actor-critic methods are Temporal Difference learning methods that have a separate memory structure to explicitly represent the policy independent of the value function. The policy structure is known as the actor, because it is used to select actions, and the estimated value function is known as the critic, because it criticizes the actions made by the actor. Learning is always on-policy: the critic must learn about and critique whatever policy is currently being followed by the actor. The critique takes the form of a TD error. This scalar signal is the sole output of the critic and drives all learning in both actor and critic as shown in the figure[28]

An actor-critic architecture maintaining a deterministic policy (actor) π , and an action-value function approximation (critic) Q . The critic's job is to approximate the actor's action-value function Q_π . Both the actor and the critic are parameterized by deep neural networks with θ_π and θ_Q , respectively. A separate copy of the actor π' and critic Q' networks are kept as target networks for stability. These networks are updated periodically using the actor π and critic networks Q modulated by a weighting parameter τ .

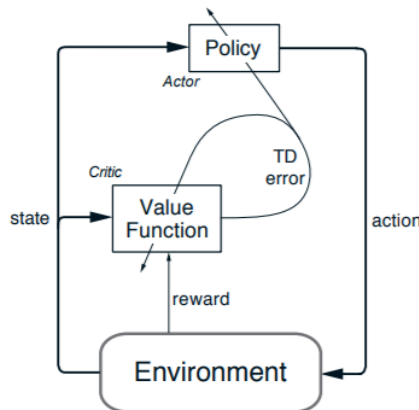


FIGURE 4.1: Basic intuition for actor critic models

Two main components in policy gradient are the policy model and the value function. It's useful learn the value function in addition to the policy, since knowing the value function can assist the policy update, such as by reducing gradient variance in vanilla policy gradients, and that is exactly what the Actor-Critic method does.

Actor-critic methods consist of two models, which may optionally share parameters: Critic updates the value function parameters w and depending on the algorithm it could be action-value $Q_w(a|s)$ or state-value $V_w(s)$. Actor updates the policy parameters θ for $\pi_\theta(a|s)$, in the direction suggested by the critic.

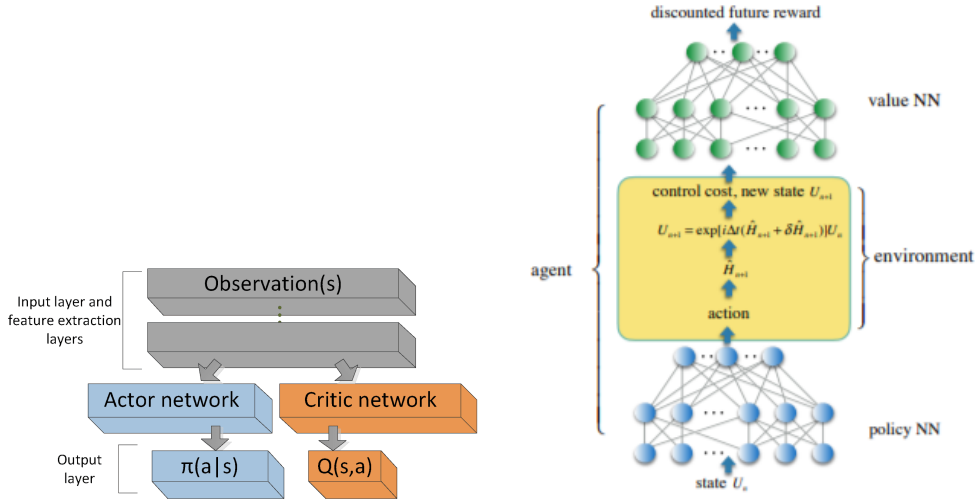


FIGURE 4.2: The workflow of actor critic models
[17]

Let's see how it works in a simple action-value actor-critic algorithm.

Algorithm 1: Actor - Critic

```

Initialize  $s, \theta, w$  at random; sample  $a \sim \pi_\theta(a|s)$ ;
for  $t = 1, 2, \dots$  until convergence do
    Sample reward  $r_t \sim R(s, a)$  and next state  $s' \sim P(s'|s, a)$ ;
    Then sample the next action  $a' \sim \pi_\theta(a'|s')$ ;
    Update the policy parameters:  $\theta \leftarrow \theta + \alpha_\theta Q_w(s, a) \Delta \theta \ln \pi_\theta(a|s)$ ;
    Compute the correction (TD error) for action-value at time  $t$ :
         $\delta_t = r_t + \gamma Q_w(s', a') - Q_w(s, a)$  and use it to update the parameters of action-value
        function:  $w \leftarrow w + \alpha_w \delta_t Q_w(s, a)$ ;
    Update  $a \leftarrow a'$  and  $s \leftarrow s'$ ;
end

```

Two learning rates, α_θ and α_w , are predefined for policy and value function parameter updates respectively.

4.3.2 Trust Region Policy Optimization

Trust Region Policy Optimization (TRPO) also belongs to the category of policy gradient. It adopts the actor-critic architecture, but modifies way the policy parameters of the actor are updated. For a new policy π' , $\eta(\pi')$ can be viewed as the expected return of policy π' in terms of the advantage over the old policy π using KL divergence (D_{KL}), which measures the difference between two distributions, here, the old and new policy.

TRPO takes into consideration a subset of region which lies within the constraint known as trust region and addresses the problem of performance improvement. By choosing the new policy parameters which maximizes the expectation subject to the KL divergence constraint, a lower bound of the expected long-term reward η is guaranteed. This also implies that we need not to worry too much about the step size with TRPO. It tries to find the optimal solution in the trust region and then starts again from there. It tries to adjust the next search region based on KL-divergence, i.e., decreases the search region if the policy is changing too much.

Algorithm 2: Trust Region Policy Optimization

```

Initialize  $\pi_0$ ;
for  $i = 0, 1, 2, \dots$  until convergence do
    Compute all advantage values  $A_{\pi_i}(s, a)$ ;
     $\pi_{i+1} = \operatorname{argmax}_{\pi} [L_{\pi_i}(\pi) - CD_{KL}^{max}(\pi_i, \pi)]$ ;
    where  $L_{\pi_i}(\pi) = \eta(\pi_i) + \sum_s \pi_i(s) \sum_a \pi(a|s) A_{\pi_i}(s, a)$ ;
end

```

4.3.3 Proximal Policy Optimization

Proximal Policy Optimization (PPO) is similar to TRPO using actor-critic architecture and uses KL divergence to decide on next policy with an additional constraint on how much we can change the policy based on KL divergence (D_{KL}).

The key contribution of PPO is ensuring that a new update of the policy does not change it too much from the previous policy. This leads to less variance and also makes sure the agent does not take unnecessary actions that leads to unsuccessful paths.

Algorithm 3: Proximal Policy Optimization

```

Initialize  $\pi_0 = \pi(\theta_0)$ , clipping threshold  $\epsilon$ ;
for  $i = 0, 1, 2, \dots$  until convergence do
    Collect partial trajectories  $D_k$  on policy  $\pi_i = \pi(\theta_i)$ ;
    Estimate advantage values  $A_t^{\pi_i}$ ;
    Compute policy update;
     $\theta_{i+1} = \operatorname{argmax}_{\theta} \mathcal{L}_{\theta_i}^{CLIP}(\theta)$ ;
    where  $\mathcal{L}_{\theta_i}^{CLIP}(\theta) = E_{\tau \sim \pi_i} [\sum_{t=0}^T [\min(r_t(\theta) A_t^{\pi_i}, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t^{\pi_i})]]$ ;
end

```

4.3.4 Reinforcement learning for temporal data

The agent used against temporal input should be able to leverage the time-dependent pattern that is present in the data. If we observe the workflow of the policy gradient methods, the nature of different policies is being captured by deep neural networks, provided the input is temporal, we can leverage the use of recurrent neural network units as a component of the policy networks and also as a component of value networks. This provides both the policy network and value network to be able to efficiently capture the pattern of the temporal data through additional information of recent event which is stored in RNN units.

Chapter 5

Methods

The decomposition of any arbitrary unitary transformation into universal gates would be inefficient because, decomposition results in large number of gates. If we could directly implement the unitary it would result in much faster gates. Here we will find the optimal pulse using Deep Reinforcement learning approach, alongside we will discuss other gradient based optimization and evolutionary algorithms like Differential Evolution, Push-Pull optimization and Evolutionary RL, along with machine learning optimizers based on sequence-to-sequence learning and search based methods like bi-directional search and Grover's search.

5.1 Differential Evolution for Quantum Control

The main aspect of the problem is the large size of the state space. The problem is a typical case of optimization with large number of control parameters. Barry Sanders et al. (2016)[6] have proposed an Adaptive Subspace based Differential Evolution Algorithm to search for an optimal pulse to implement Quantum gates. We discuss their methods and present our attempts at reproducing their results.

5.1.1 Differential Evolution Overview

Differential Evolution (DE) is part of generic population-based meta-heuristic optimization algorithms. It assumes no prior knowledge about the search space and is very effective in searching large state spaces. But it does not guarantee convergence to the optimal solution.

A standard DE algorithm has 3 main steps. We start by spawning P number of candidate solutions that we call the population. For each member of the population a child is created through crossover and mutation. The fittest of the parent and the child is then chosen. This basically implies the survival of the fittest.

1. Mutation: Random genes are inserted in offspring to maintain the diversity in population to avoid premature convergence
2. Crossover: This is the step where the daughter is created. Parent members of the population are chosen and then "combined" to give a daughter offspring
3. Selection: Here the fittest of the parent and the offspring is chosen

5.1.2 SUSSADE

SUSSADE proposed by Barry Sanders et al. (2016)[6] stands for Subspace-selective self-adaptive differential evolution. We examine this in a bottom up approach by first looking at barebones DE. Explicitly the three steps in DE are as follows

1. Mutation: Each population member has a child assigned by choosing three random parents D_1, D_2 and D_3 . The daughter is $M = D_1 + \mu(D_2 - D_3)$ where μ is a random number uniformly sampled from $[0, 1]$. A higher value of μ means the child is more different from the parent as compared to when μ is smaller.

2. Crossover: Each population member has K control parameters which we call chromosomes and the child either retains its chromosome or receives the parent chromosome with probability ζ .
3. Selection: In the selection step, the fittest of the child and the parent is chosen to become the parent for the next generation.

Now let's look at the self adaptivity. If the parameters μ and ζ are updated as the evolution progresses we can potentially have faster convergence and a better solution.

$$\mu_{i,G+1} = \mu_l + r_1 \mu_u \quad \text{if } r_2 < \kappa_1; \quad \mu_{i,G} \quad \text{otherwise}$$

and

$$\zeta_{i,G+1} = r_3 \quad \text{if } r_4 < \kappa_2; \quad \zeta_{i,G} \quad \text{otherwise}$$

where r_j are random numbers uniformly sampled from $(0, 1]$ and μ_l, μ_u, κ_1 and κ_2 are assigned to fixed values of 0.1, 0.1, 0.1 and 0.9. The index i denotes the population member and G denotes the generation.

The last modification is subspace selectivity in which instead of breeding and evolving the entire space, a subspace or particular genes are selected and then evolved. For the target matrices used in Barry Sanders et al. (2016)[6] the trivial sub-space of size 1 is sufficient.

5.2 Push Pull optimization

The core of an optimization algorithm is an objective function which evaluates the overlap of the realized process with the target process and hence should be chosen carefully. One such hybrid objective function that not only depends on the target operators but also on the set of orthogonal operators is suggested. Utilizing which, the control parameters can be pulled towards the target operator and at the same time pushed away by the orthogonal operators. This approach is called Push-Pull Optimization of Quantum Controls[21] (PPOQC).

The gate control or state control for a d -dimensional target operator to be evaluated utilizing the $d-1$ orthogonal operators obtained via Gram-Schmidt orthogonalization process. These orthogonal operators push the control-sequence away from them and target operator does the opposite. So, we can define Push fidelities as,

$$GC : F_o(U_{1:N}, V_l) = \frac{1}{L} \sum_{l=1}^L F(U_{1:N}, V_l) \quad SC : F_o(\rho_{1:N}, R_l) = \frac{1}{L} \sum_{l=1}^L F(\rho_{1:N}, R_l)$$

where V_l and R_l are $L \leq d - 1$ orthogonal operators such that $F(U_t, V_l) = 0$ and $F(\rho_t, R_l) = 0$.

Of course, d increases exponentially with the system size, but as we shall see later, a small subset of L orthogonal operators can bring about a substantial advantage. Also, note that for a given target operator, the set of orthogonal operators is not unique and can be generated randomly and efficiently in every iteration. We define the push-pull performance function

$$J_{PP} = F - \alpha F_o - \sum_{k=1}^M \lambda_k r_k,$$

where $-1 \leq \alpha \leq 1$ is the push weight.

The main objective of this method is to bring down the error in the optimal solution. the degree of closeness that is improved at each step is higher than traditional method because of the push away effect from the orthogonal operators.

This method takes computationally the same time as that of traditional method to converge as the number of steps to get closer to the solution in the given target operator direction is same.

5.3 RNNs - sequence to sequence learning

This method is inspired from the similarity between the control problem and sequence-to-sequence based problems, as discussed in 3.6. We leverage the advantage provided by RNNs to store the information about timed data. We use encoder decoder architecture to learn the relation between input sequence to optimal pulse sequence. We use the most common activation function for RNNs, tanh. Based on the complexity of the problem, the network is scaled accordingly. Hyper-parameter tuning is done using tensorflow keras tuner. More details about the architecture used is provided in next chapter.

5.4 Search based methods

Control problem can be formulated as a search problem, where the optimal sequence is obtained by searching in the feasible region. The only problem is the length of sequence should be fixed and each value in the sequence should be from a discretized range. As we can see, the search space grows exponentially as we increase the length of the sequence.

5.4.1 Grover's search

Grover's algorithm, also known as the quantum search algorithm, refers to a quantum algorithm for unstructured search that finds with high probability the unique input to a black box function that produces a particular output value, using just $O(\sqrt{N})$ evaluations of the function, where N is the size of the function's domain. This method works when the number of values satisfying the conditions of the search are very less compared to the total size of search space.

There are two main parts to the algorithm,

- The input to the system is superposition over all the states. Create a quantum Oracle which can mark the states that satisfy the conditions of the search. By marking, we mean, the states in the input which satisfy the conditions is flipped and those doesn't stay the same.
- After getting the marked values, double the values of the marked states, there by increasing the probability of these marked states, and re-normalize the probabilities of the states such that it sums to 1.

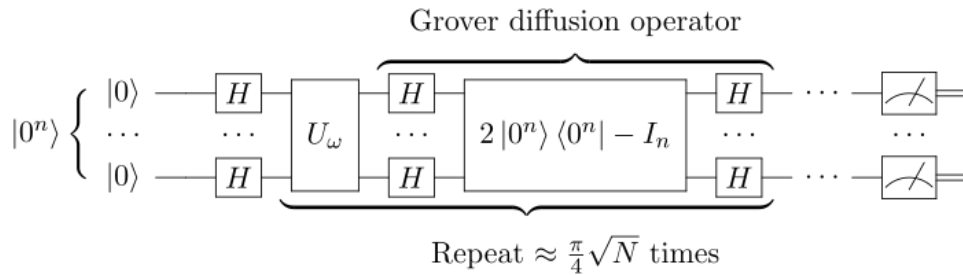


FIGURE 5.1: Outline of Grover's Search

To describe clearly about the quantum oracle, unitary operator U_{opt} that acts as follows, where $f(x) = 1$, implies the conditions of the search is satisfied and not if otherwise, this can be understood as reflection of desired states:

$$\begin{aligned}
 U_{opt}|x\rangle &= -|x\rangle \quad \text{for } x, \text{ that is, } f(x) = 1, \\
 U_{opt}|x\rangle &= |x\rangle \quad \text{for } x, \text{ that is, } f(x) = 0.
 \end{aligned}$$

The diffusion operator $U_{diffusion} = 2|s\rangle\langle s| - I$, where $|s\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle$ gives, negative of the value if $f(x) = 1$, and same value if $f(x) = 0$, because, the states are orthonormal to each other and $f(x)=1$ implies $|x\rangle$ is changed to $-|x\rangle$. This can be understood as reflection about the average amplitude.

Algorithm 4: Grover's Search

```

Initialize the system to the uniform superposition over all states  $|s\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle$ ;
//Perform the following "Grover iteration" r(N) (upper limit on iterations) times;;
for  $i = 0, 1, 2, \dots, r(N)$  do
    | Apply the operator  $U_{opt}$ ;
    | Apply the Grover diffusion operator  $U_{diffusion} = 2|s\rangle\langle s| - I$ ;
end
Measure the resulting quantum state in the computational basis.;

```

The difficult part of the solution is designing the quantum oracle which can mark the states. For the control problem, with discretized pulse sequence of fixed length, the sequence can be encoded into a binary string which can be represented as superposition of quantum states and the optimal sequence needs to be searched in this search space.

The core idea of this method relies upon construction of the quantum oracle. Provided we have a quantum oracle that marks bang-bang control sequences if the fidelity for that sequence is above a certain value, we can try to build a modular repetitive system that effectively find the optimal sequence in the search space. The modular system would encode all possible bang-bang control sequences into the superposition over all states and search among these states. For a sequence of length N , we need N qubits, if rather than bang-bang control, if we have p -level control system, then we need $\log_2 p^N$ qubits.

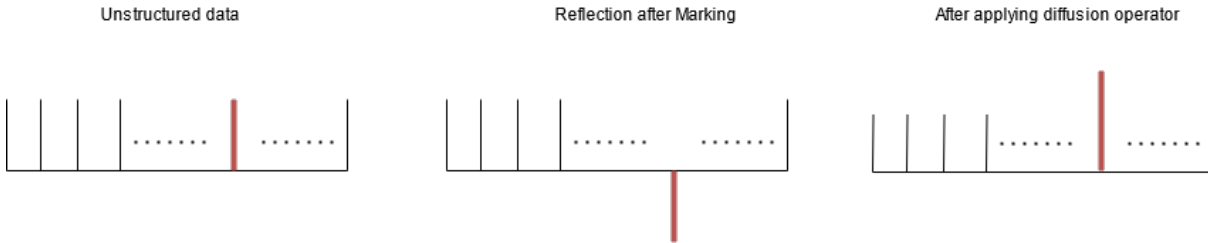


FIGURE 5.2: Grover Iteration

We have four possibilities from here on, Let's say we are trying to control a N_1 qubit system with N_2 qubit controller used for Grover's search.

- If $N_1 = N_2$, the system can self-optimize.
- If $N_1 < N_2$, we need larger system to optimize smaller system.
- If $N_1 > N_2$, then we need smaller system to optimize.
- for any N_1 , if constant value N_2 (N^*), a fixed system is needed to optimize.

But mathematically, we can always concatenate more modular units to increase the length of control sequence to achieve better fidelity. We are still trying to solve this problem mathematically, to achieve such consensus.

5.4.2 Bi-directional search

Most of the search algorithms will be either search forward, starting from the initial state based on a heuristic of its own state alone or search backwards, starting from the target state, trying to reach initial state based on some heuristic. If the possibilities to move to next state, multiply by at-least a

factor k ($k > 1$) after approaching any given state, then the search space grows exponentially. Let's say the optimal solution is of the length p , then total number of paths to search would be exponential in p , but if the search starts from either directions, and if it is known that they meet in the middle, then the length becomes half, so the search space would become exponential in $p/2$, which is similar to square-root speed-up.

The core idea relies on the guarantee that every sequence starting at initial sequence needs to end in the target state eventually. So far, even the control problem we are trying to solve, will provide this guarantee, provided there is a precision ball around the target state. This search algorithm can also be perceived as two points having their initial points as the start state and target state, trying to approach each other based on a heuristic that depends on the states of each other.

5.5 Deep Reinforcement learning for quantum control

We try to find the optimal sequence for the following quantum control problems using Trust Region Policy Optimization (TRPO). We also critic on Proximal Policy Optimization (PPO). In the following sections, we will describe about the network we have used and the hamiltonian we have used for each of the single-qubit, two-qubit and three-qubit gates

5.6 Implementation Details

We have used stable baselines (version 2.10.0) package to implement the TRPO and PPO algorithms. The environment is implemented using gym library. Tensorflow for building neural network architecture and qutip for simulating physical systems. The common details for the following implementations are,

- The modelling of physical system is done using gym which have discrete action values for bang-bang control in single and two-qubit gates and continuous action values for three-qubit gates and the fidelity calculations are key bottle-neck for the performance.
- Google Colab, which provides 12GB RAM and 1TB storage space is used as the backend for running the code and is taking approximately 15min, 1hr and 3.5hrs for running 15000 episodes for single-qubit, two-qubit and three-qubit respectively.

In the bang-bang control protocol, the total time T is fixed and it is divided into N time intervals with same duration $\delta t = \frac{T}{N}$. The control parameter $\epsilon_i(t)$ is constant in the i^{th} time interval where it takes either +range limit or -range limit. Where as in the continuous action value protocol, the control parameter is interpolated from $\epsilon_{i-1}(t)$ to $\epsilon_i(t)$

The Unitary Evolution, given by $\mathcal{U}(i\delta t, (i-1)\delta t; \epsilon_t(i))$ where $\epsilon_0, \epsilon_1, \dots, \epsilon_N$ are N control parameters.

The aim is to obtain the optimal parameter vectors $\epsilon_1, \epsilon_2, \dots, \epsilon_N$ to make the unitary operator $\mathcal{U}(T)$ as close to the target unitary gate $\mathcal{U}_f$ as possible, which is formulated as maximizing fidelity $\mathcal{F}(T)$ at T , say $\epsilon(t) = \epsilon_1, \epsilon_2, \dots, \epsilon_N$, then,

$$\mathcal{U}(i\delta t, (i-1)\delta t; \epsilon_t(i)) = e^{\frac{-i\mathcal{H}(\epsilon(t))\delta t}{\hbar}} \quad (5.1)$$

$$\mathcal{U}(i\delta t) = \mathcal{U}(i\delta t, (i-1)\delta t; \epsilon_t(i))\mathcal{U}((i-1)\delta t); \mathcal{U}(0) = \mathcal{I} \quad (5.2)$$

where $\mathcal{I}$ is the identity operator of Hilbert Space.

$$\mathcal{F}(T) = \max_{\epsilon(t)} \mathcal{F}(T; \epsilon_1, \epsilon_2, \dots, \epsilon_N) \quad (5.3)$$

$$\mathcal{F}(T; \epsilon_1, \epsilon_2, \dots, \epsilon_N) = \left| \frac{\text{Tr}(\mathcal{U}_f^\dagger \mathcal{U}(T))}{D} \right|^2 \quad (5.4)$$

where D is the dimension of the Hilbert Space and $\text{Tr}\{.\}$ is trace of the matrix.

5.7 Physical Systems

The fault-tolerant quantum computing devices can be implemented using various physical systems, few of which are, Ion traps used by IonQ and superconducting qubit system used by IBM. The superconducting qubit system are fast gates with unparalleled combination of good coherence and flexibility in design parameters. Most of the demonstrations of small scale quantum algorithms in superconducting architecture used transmon qubits and cross-qubit coupling.

5.7.1 Cross-Resonance Gate

The long term goal of building quantum circuits is to scale the system large enough such that we would be able to implement large-scale algorithms with high-accuracy. Interestingly, a pair of superconducting qubits with resonator-mediated interaction between them, also called as cross-resonance gate has been implemented experimentally with less than 1% average infidelity, providing necessary promise for scalability.

In the cross-resonance gate, the gates are functioning in such a way that a given qubit is driven at the resonant frequency of the other qubit, inducing the dynamics of the latter through the connecting resonator and hence the name, cross-resonance. It is an entangling gate for fixed frequency superconducting qubits. These are comparatively slow but easily extensible.

We use the system similar to the system used by Vijay et.al[5], and is as shown below,

$$\frac{\mathcal{H}}{\hbar} = \mathcal{H}_{drift} + \mathcal{H}_{control}$$

$$\mathcal{H}_{drift} = \sum_{i=1}^2 [\omega_i a_i^\dagger a_i + \frac{\delta_i}{2} a_i^\dagger a_i (\mathcal{I} - a_i^\dagger a_i)] + J(a_1^\dagger a_2 + a_2^\dagger a_1)$$

$$\mathcal{H}_{control} = \sum_{i=1}^2 \mathcal{A}_i(t) \text{Cos}(\omega_{a,i}t + \phi_i(t)) [a_i^\dagger + a_i]$$

$$\omega_i \longrightarrow \sim 5\text{GHz}, \quad \text{detuned by } 100 - 200\text{MHz}$$

$$\delta_i \longrightarrow \sim 300\text{GHz}$$

$$J \longrightarrow 2 - 15\text{MHz}$$

$$\mathcal{A}_i \longrightarrow 20 - 60\text{MHz}$$

5.7.2 Single-qubit

As given in the paper by Zheng An et.al, the two-level system whose Hamiltonian is

$$\mathcal{H}(\epsilon(t)) = \sigma_z + \epsilon(t)\sigma_x \quad (5.5)$$

where σ_z, σ_x are Pauli matrices and $\epsilon(t)$ is a real control parameter, is used. This model had been used for in quantum physics, like in the Landau-Zener-Stuckelberg interferometry and the Kibble-Zurek mechanism.

The control paramter $\epsilon(t)$ can take either +4 or -4 in our bang-bang protocol. The target quantum gate is Hadamard gate,

$$\mathcal{U}_f = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

5.7.3 Two-qubit

The four-level system whose Hamiltonian is

$$\mathcal{H}(\epsilon(t)) = \sigma_z \otimes \sigma_z + \epsilon_1(t)\sigma_x \otimes \mathcal{I} + \epsilon_2(t)\mathcal{I} \otimes \sigma_x + \epsilon_3(t)\sigma_y \otimes \mathcal{I} + \epsilon_4(t)\mathcal{I} \otimes \sigma_y \quad (5.6)$$

where $\sigma_z, \sigma_y, \sigma_x$ are Pauli matrices and $\epsilon(t) = \{\epsilon_0(t), \epsilon_1(t), \epsilon_2(t), \epsilon_3(t)\}$ is a real control parameter, is used. Each of the control parameter $\epsilon_i(t)$ can take either +4 or -4 in our bang-bang protocol. The target quantum gate is CNOT gate,

$$\mathcal{U}_f = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

5.7.4 three-qubit

The Hamiltonian is borrowed from the paper by Barry Sanders et. al,[6], the implementation is proceeded as given in the section 2.4.2. The hamiltonian is also shown below,

$$\frac{\hat{\mathcal{H}}(t)}{h} = \sum_{k=1}^n \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & \epsilon_k(t) & 0 & 0 \\ 0 & 0 & 2\epsilon_k(t) - \eta & 0 \\ 0 & 0 & 0 & 3\epsilon_k(t) - \eta' \end{pmatrix}_k + \frac{g}{2} \sum_{k=1}^{n-1} (\hat{X}_k \hat{X}_{k+1} + \hat{Y}_k \hat{Y}_{k+1}) \quad (5.7)$$

The target quantum gate is CCZ gate,

$$\mathcal{U}_f = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 \end{pmatrix}$$

The other target gates are Toffoli and Fredkin gates. The Toffoli and Fredkin gates are almost diagonal and is sparse matrix. The main challenge being to find optimal sequence for $\mathcal{U}_{fullmat} = \mathcal{H} \otimes \mathcal{H} \otimes \mathcal{H}$, where $\mathcal{H}$ is hadamard gate. This would be a full matrix and trying to optimize the sequence to match all these values would be relatively harder.

Chapter 6

Results

6.1 Simulation results for single-qubit

In both the methods, Proximal Policy Optimization and Trust Region Policy Optimization, the length of the control sequence is varied from 20 to 100 in steps of 10. The policy and value network uses Multi-layer-Perceptron(Mlp) policy of 2 layers of 64 each as the hidden layers for each pulse being either +4 or -4. The target Unitary is Hadamard gate. The key aspect is that for the model is able to stabilize after certain iterations and is able to adapt to different landscapes which varies accordingly with target gate.

6.1.1 Comparison with results in paper by Marin Bukov et. al

Marin Bukov had used Q-learning to find the optimal sequence in 20000 episodes, we here present you TRPO method which had obtained optimal sequence with fidelity 0.99987 and convergence is observed at around 5000 episodes.

6.1.2 Proximal Policy Optimization

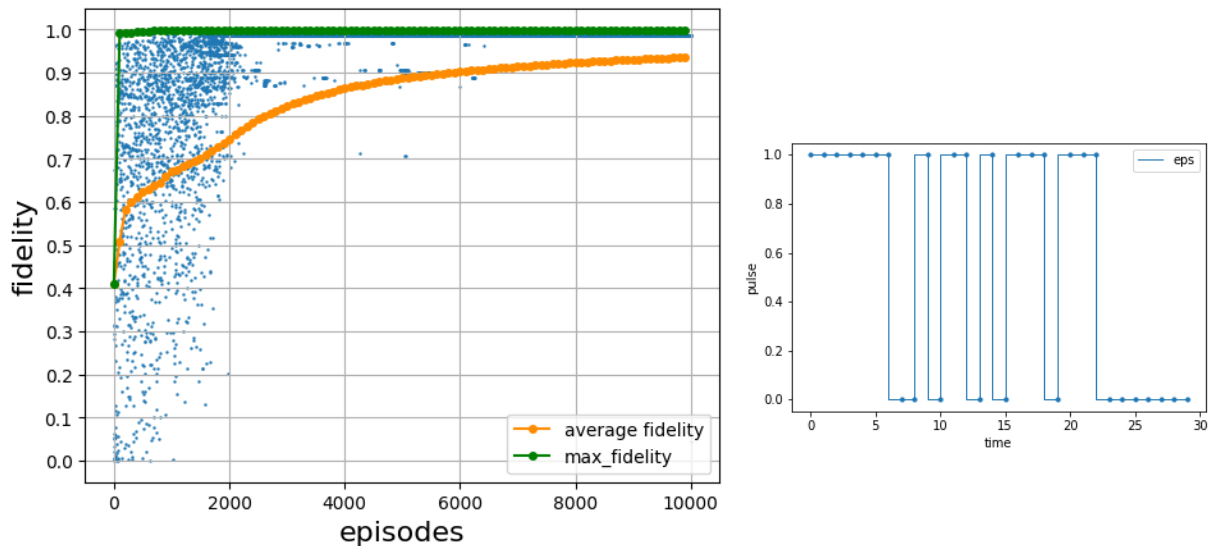


FIGURE 6.1: Proximal Policy Optimization for single qubit system. The above shows fidelity vs episodes and corresponding pulse sequence. Convergence achieved at around 5000 episodes with max fidelity as 0.9993.

6.1.3 Trust Region Policy Optimization

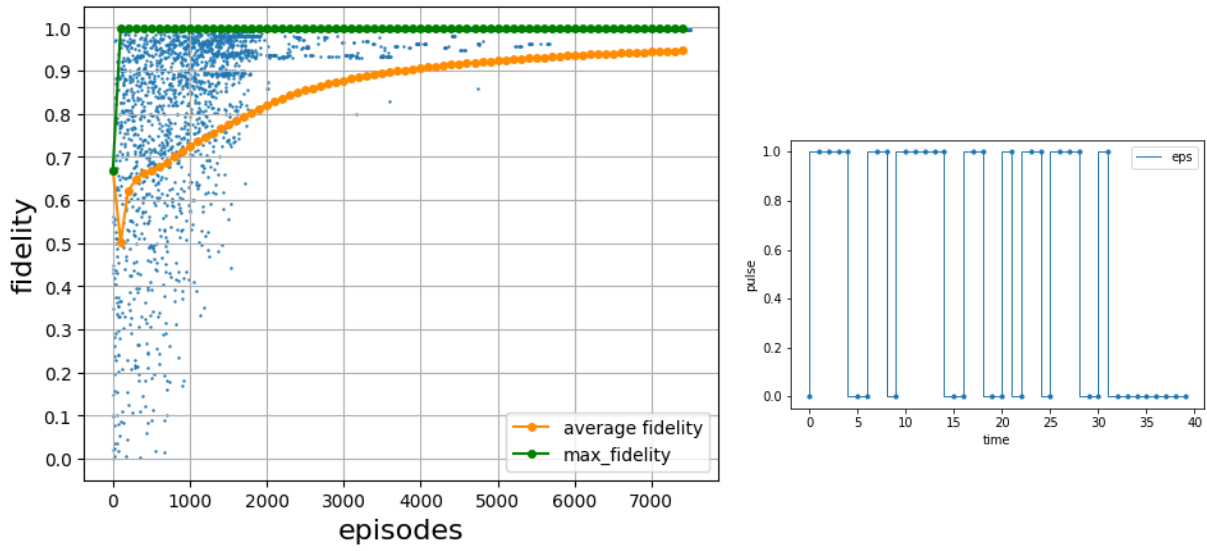


FIGURE 6.2: Trust Region Policy Optimization for single qubit system. The above shows fidelity vs episodes and corresponding pulse sequence. Convergence achieved at around 4000 episodes with max fidelity as 0.99987.

6.1.4 Variation across length of pulse sequence

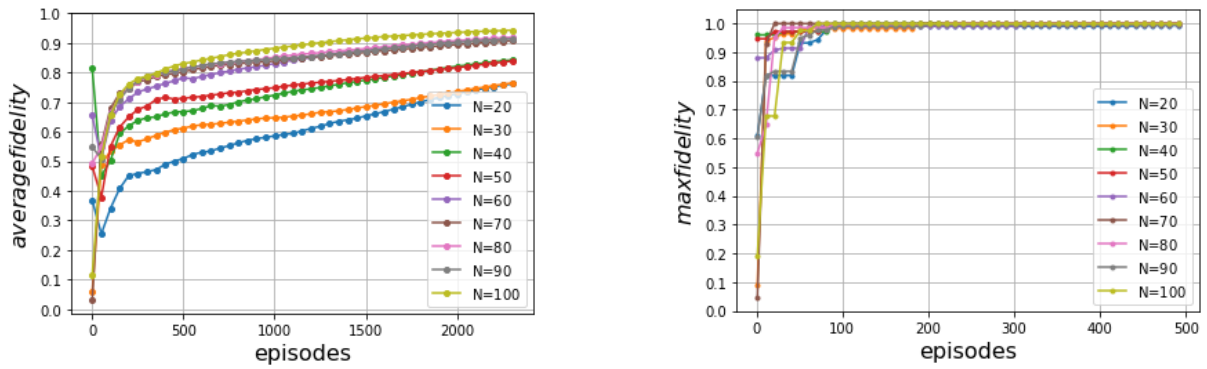


FIGURE 6.3: Variation of average and maximum fidelity with respect to length of the pulse sequence

In the Figure 6.3, we can see that increasing the length of the pulse sequence increases the average fidelity achieved in the fixed episodes, but the finding the sequence with maximum fidelity might take longer because of its larger search space.

6.2 Simulation results for two-qubit

In both the methods, Proximal Policy Optimization and Trust Region Policy Optimization. The length of the control sequence is varied from 20 to 100 in steps of 10 timesteps. The policy and value network uses Multi-layer-Perceptron (Mlp) policy of 3 layers of 600 each as the hidden layers for each pulse being either +4 or -4. The target unitary is CNOT.

6.2.1 Comparison with results in paper by Zheng An et. al

Zheng An had used DDDQN to find the optimal sequence in 17500 episodes, we here present you TRPO method which had obtained optimal sequence with fidelity 0.9993 and convergence is observed at around the same number of episodes. But the training time is relatively less due to increased number of hidden layers of value network

6.2.2 Proximal Policy Optimization

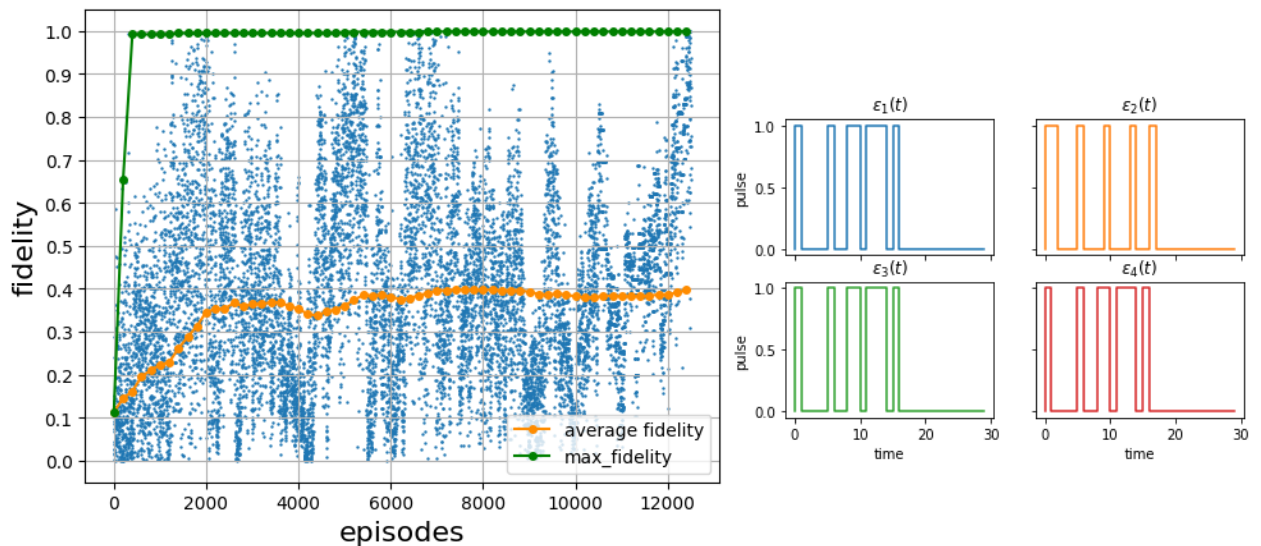


FIGURE 6.4: Proximal Policy Optimization for two qubit system. The above shows fidelity vs episodes and corresponding pulse sequence. Max fidelity as 0.9933.

6.2.3 Trust Region Policy Optimization

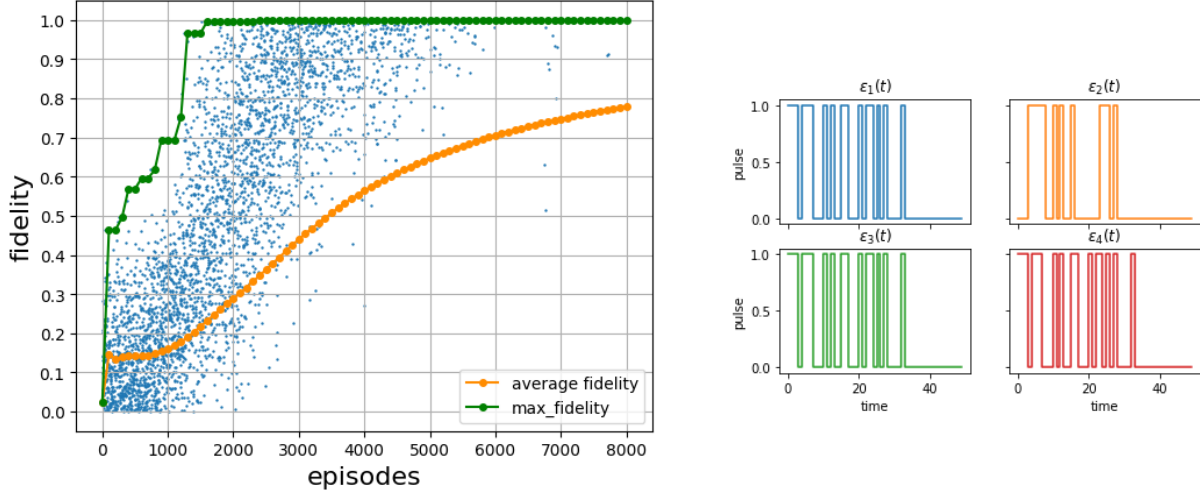


FIGURE 6.5: Trust Region Policy Optimization for two qubit system. The above shows fidelity vs episodes and corresponding pulse sequence. Convergence achieved at around 4000 episodes with max fidelity as 0.9993.

6.3 Simulation results for three-qubit

To implement deep reinforcement learning, we are using stable baselines package. The underlying physical system is three-qubit transmon system used in Barry Sanders et. al., paper and we tried to optimize CCZ gate. The length of the control sequence is 30 timesteps with continuous action value protocol. The policy and value network uses Multi-layer-Perceptron(Mlp) policy of 5 layers of 600, 600, 600, 300, 600 as the hidden layers for each pulse being in the interval $[0.4, -0.4]$. The CCZ gate is used to compare the performance between PPO and TRPO. Then we show our results for Toffoli gate and Fredkin gate using TRPO, accounting for sparse matrix which is not completely diagonal.

6.3.1 Comparison with results in paper by Barry Sanders et. al

The process works well for less dense matrix, but the TRPO method is flexible and we can input different physical systems to the existing TRPO architecture and obtain the results, making it flexible to use.

The results with Toffoli gate and Fredkin gate shows that TRPO performances good with sparse matrices and with dense matrices, the performance is not as expected for the given network.

6.3.2 Proximal Policy Optimization

We were able to obtain a fidelity of 0.9927 in 17500 episodes. We have assumed coherence throughout the process. The issue here is clipping with respect to KL divergence didn't allow the agent to explore new regions easily, thereby limiting the performance.

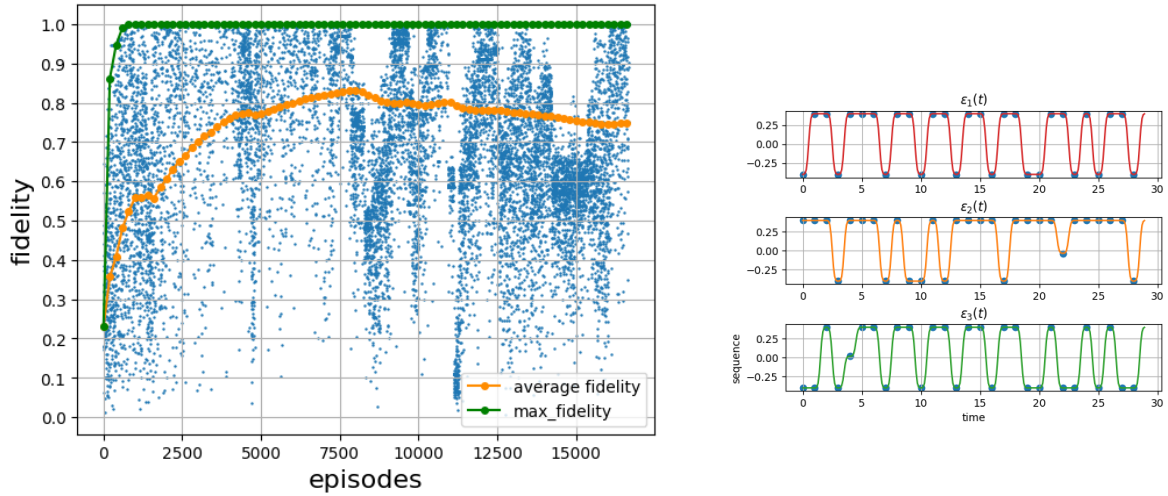


FIGURE 6.6: Proximal Policy Optimization for three qubit system. The above shows fidelity vs episodes and corresponding pulse sequence. Max fidelity as 0.9927.

6.3.3 Trust Region Policy Optimization

We were able to obtain a fidelity of 0.999770 in 17500 episodes. We have assumed coherence throughout the process.

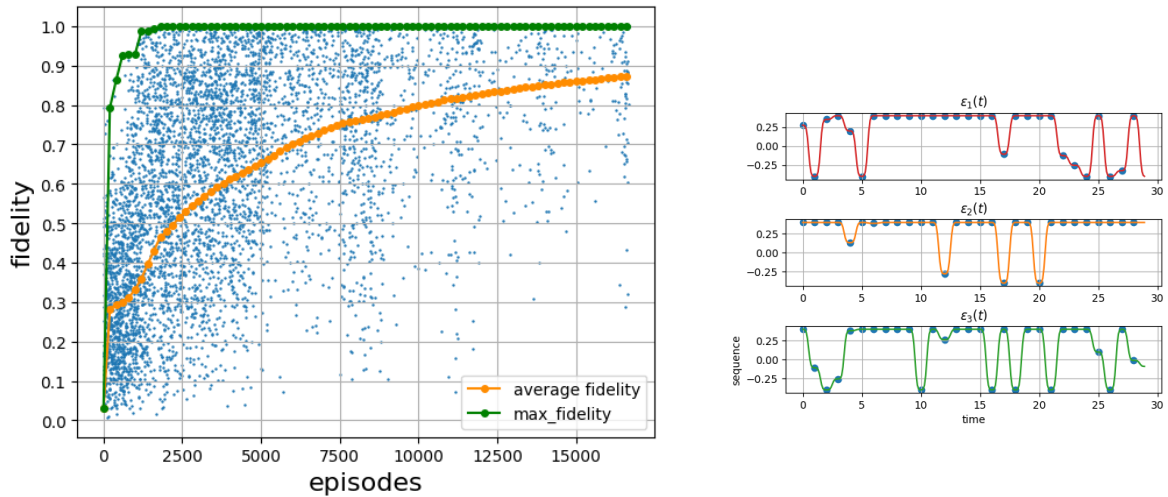


FIGURE 6.7: Trust Region Policy Optimization for three qubit system. The above shows fidelity vs episodes and corresponding pulse sequence. Convergence achieved at around 15000 episodes with max fidelity as 0.999770.

Results of TRPO with Toffoli and Fredkin are as shown in 6.8, 6.9 respectively.

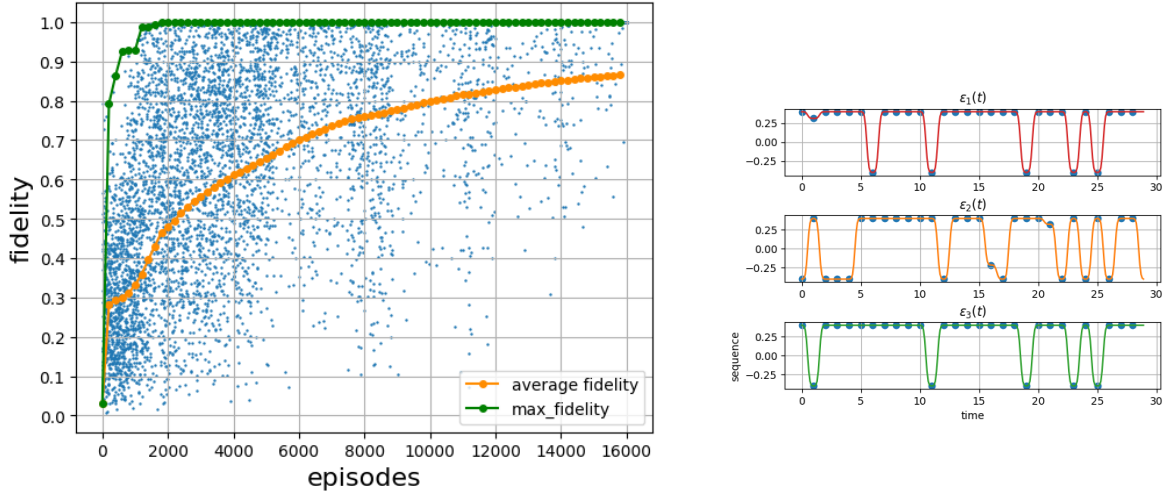


FIGURE 6.8: Fidelity vs. number of episodes for Toffoli Gate. convergence is achieved at around 14000 episodes. Highest fidelity is 0.9993.

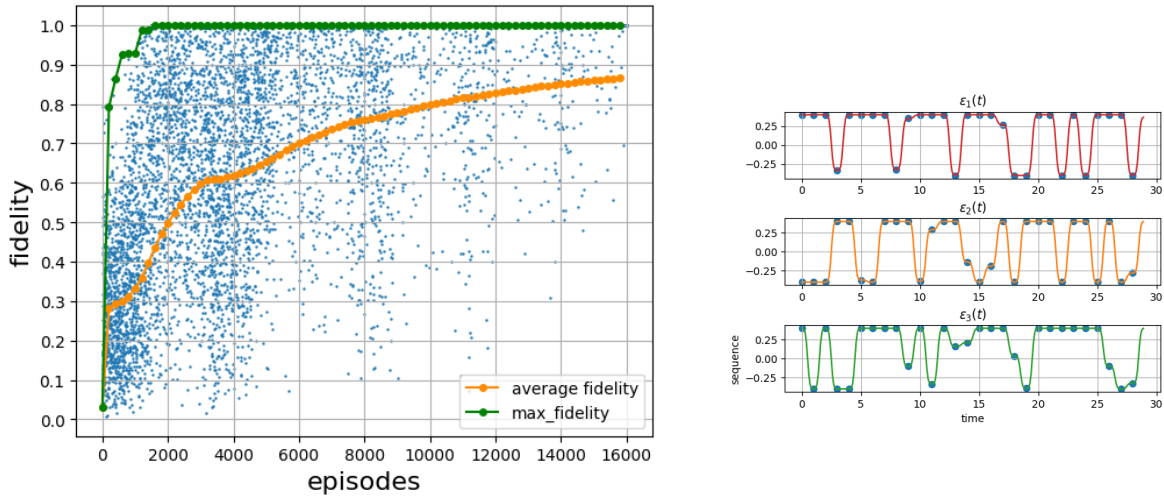


FIGURE 6.9: Fidelity vs. number of episodes for Fredkin gate. convergence is achieved at around 15000 episodes. Highest fidelity is 0.9995.

6.4 Simulation results for two-qubit Cross Resonance Gate

6.4.1 RNN method

We implemented the encoder-decoder architecture with decreasing LSTM units from 150 to 50 in steps of 50 with repeated vectorization equals to the discretization of pulse sequence and decoder is just inverse of this pattern. Before we pass it through this network, we add a 1D-convolutional layer with activation function as ReLU. We use the Cross-Resonance Gate system as shown in 5.7.1.

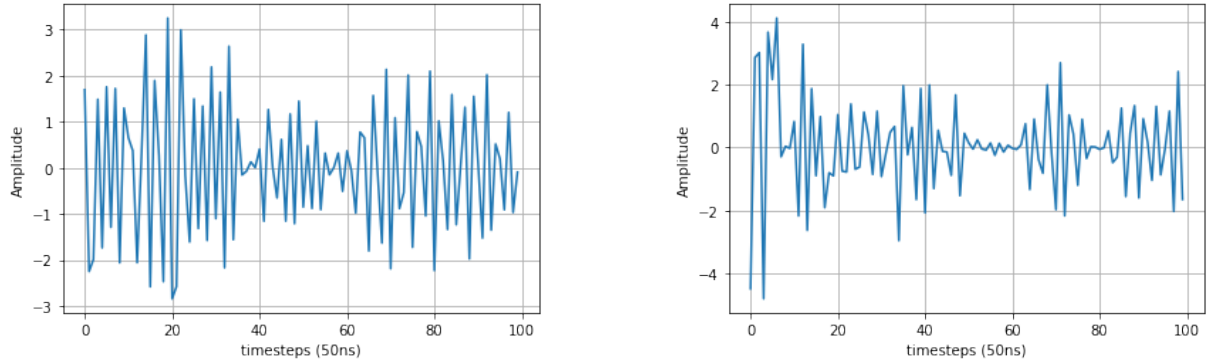


FIGURE 6.10: Amplitudes of the two qubits respectively for the cross resonance gate with fidelity as 0.99996 for CNOT target gate.

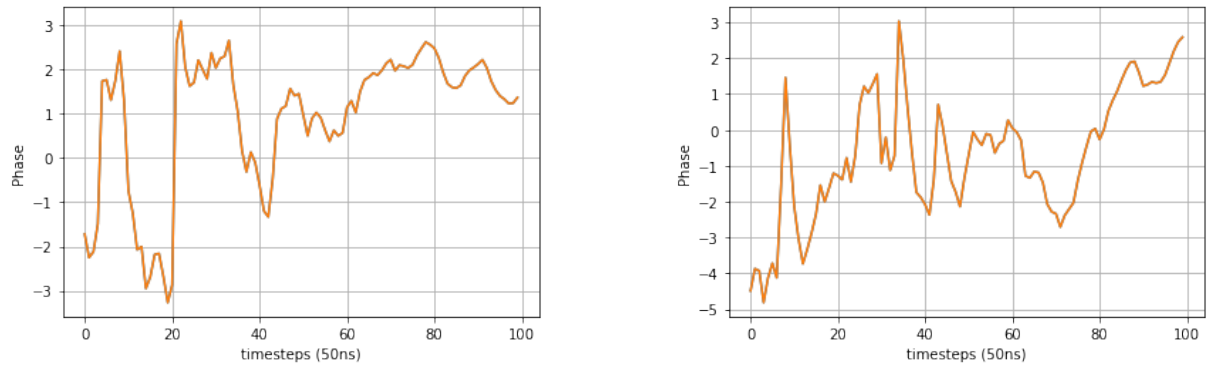


FIGURE 6.11: Phases of the two qubits respectively for the cross resonance gate with fidelity as 0.99996 for CNOT target gate.

The 2-qubit system is 3-level system, each qubit able to take values from 0, 1, 2. The first qubit frequency is 5GHz and that of second qubit is 5.2 GHz. They are detuned by 300MHz and coupling factor of 15MHz. The loss function is infidelity, initially, but with this, the amplitudes are fluctuating a lot between significantly high values. We have to modify the loss function by adding l2-norm of the amplitudes to restrict them to oscillate between smaller values.

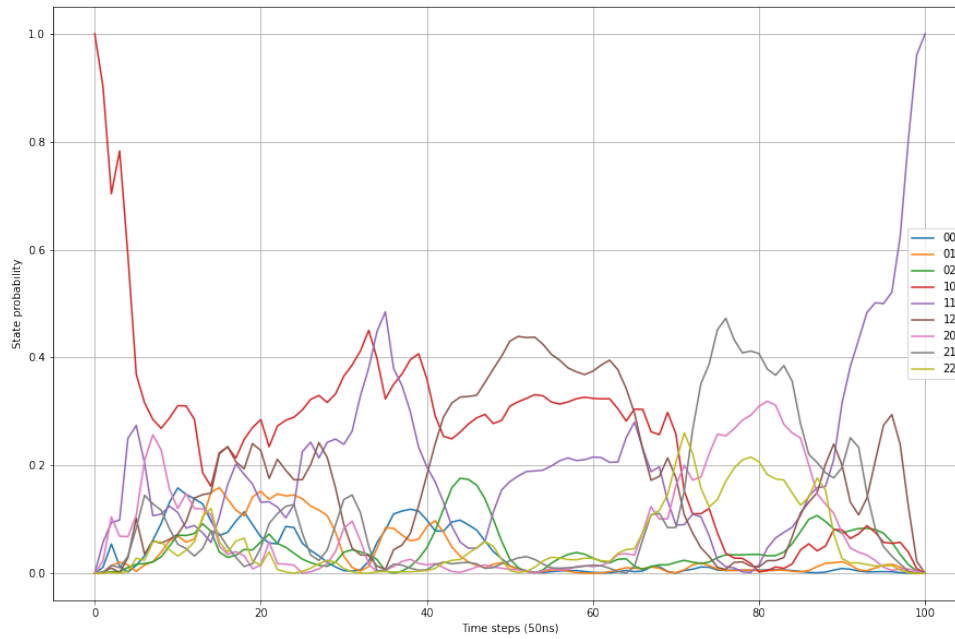


FIGURE 6.12: The state transition for CNOT

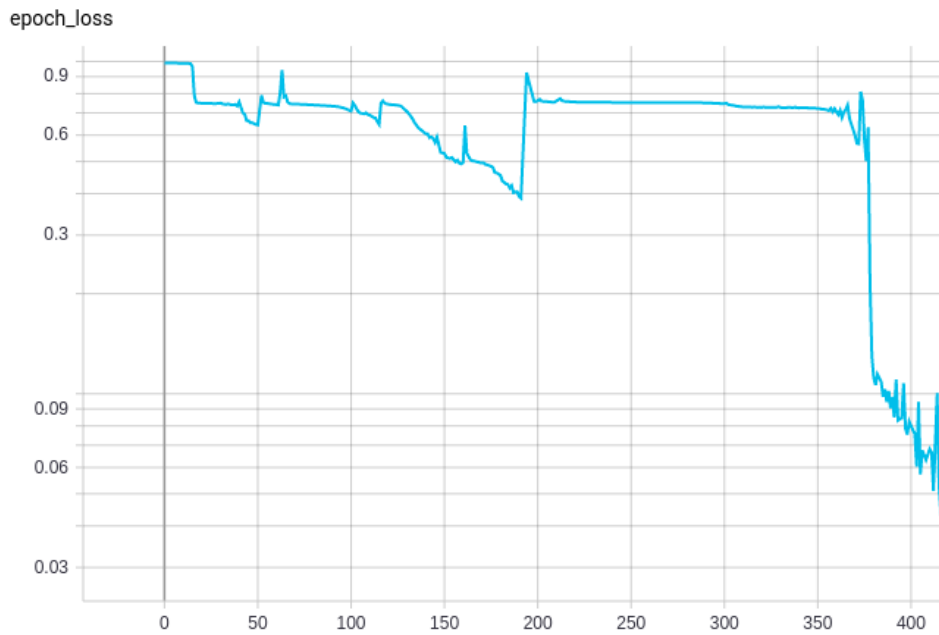


FIGURE 6.13: Loss value which is Infidelity goes down to 0.03 during training cross resonance gate for QFT as the target gate

Chapter 7

Discussion about scalability and performance

7.1 Comparison of methods

Both RNN methods and Deep RL methods, are able to find optimal solutions, but training and hyper-parameter tuning of the RNN-based methods are a bit tedious. While the RL methods are easier to train but, were not able to perform as expected when the target matrix is relatively dense. Both these methods have pros and cons, where the RL methods are able to perform amidst errors/noise in the system, but when the search space increases exponentially, they are not performing as good as expected, while RNN methods require fixed length for input and output sequence but are able to provide very good results even for the cases where the search space is larger.

More about RL methods, both TRPO and PPO are able to find optimal solutions in the single-qubit system and two-qubit system. But in three-qubit system, PPO finds trouble finding the optimal solution where as TRPO continues to perform well. The reason being, PPO restricts the exploration by limiting the extend of deviation from the old policy based on KL divergence which is not the case with TRPO.

7.2 Performance & how it varies

The key elements in the above problem are,

- Depth of the neural network
- Exploration space by increasing the range of values that can be taken by the pulse
- training time and resources used

The performance of deep RL methods, depends on the depth of the neural network that is being used. The policy network and value network are both deep neural networks and based on the exploration space, it should be altered. The neural network can be considered as a function estimator where the adjustments are made based on the input and output for each connection in the network every time we train the network. Having a deeper neural network might over-fit the function and hence not perform well and having a shallow neural network will not have sufficient connections to create appropriate function estimator. Managing the trade-off is key to come up with appropriate network that performs well. The performance of the RNN method, depends on the constraint on the length of the pulse sequence, because of the difficulty to adjust the parameters to train the model. The encoder-decoder architecture should be altered for any changes in the underlying physical system and hyper-parameter tuning should also be done everytime we can change the physical system.

As the exploration space increases with increase in number of qubits, the depth of the network should be increased accordingly to get results quickly. As we can see, the larger exploration space will have more constraints and hence need more connection in the network to learn. Also, the ability to adapt to errors in higher qubit systems, will be dependent on the depth of the network, needing deeper policy and value networks for RL methods and more number of encoder-decoder architectures concatenated.

Number of qubits doesn't effect the RL module or RNN module, given the control knobs (the length of pulse sequence) kept constant, but increasing number of qubits will cause increase in computational effort to compute fidelity (the reward value in this case) thereby requiring us to paralleling the computation using most efficient methods.

7.3 Scalability

We have seen multiple solutions like, gradient based optimization which depends on initial sequence and which doesn't depend on initial sequence, along with, machine learning methods which require labelled data or un-labelled data to understand the patterns in the data, and reinforcement learning methods which is data-intensive and requires data on demand to learn the underlying dynamics. When we think about the scalability of the solution, we can discuss about each method and a combination of these methods. Individually, each of the methods have specific reasons and might not be able to find the solutions with required accuracy and precision.

The gradient methods will not be able to converge to global optimum if the underlying landscape is ragged or values varies rapidly. But the data of good and bad pulse sequences can be generated easily. The machine learning approach using RNNs becomes training-exhaustive as the number of the qubits increases, because for bigger systems, we would be requiring larger pulse sequences which requires deeper networks increasing the training time and requiring new data if we want to build better networks. The RL methods are robust to the noise or disturbances in the physical system but the convergence might require deeper networks to train and efficient reward functions to incorporate bounding conditions on the sequences and speed up the training.

The deep RL methods can be made model-free by choosing appropriate definition for control sequence, most importantly, the reward function associated with the method. Given a computational model for the underlying physical system which can compute the fidelity of a given sequence in time that is polynomial function in number of qubits, the deep RL methods will converge in polynomial time using appropriate depth of the neural network used in the method.

So, the hybrid solution would be a better scalable solution for the problem, where the optimizer is mixture of all the above proposed methods, i.e, the system is given to the optimizer, which will be run past gradient optimizer, initially, which keeps track of both good and bad pulse sequences creating the data required for the machine learning approaches. The machine learning optimizer produces better sequences, giving a better initial sequence for gradient optimizers which are sensitive to start sequence. The RL based optimizer can also be used to produce better start sequence for the gradient optimizers. This is considered as a iteration for the hybrid solution. In the next iteration, the start sequence is provided to the gradient optimizer, which in-turn be looped to machine learning or RL optimizers until convergence is achieved. The other possible way to scale the solution is to check if the optimizers for smaller qubit systems can be combined to solve larger qubit systems. The implementation is shown in section A.2

Chapter 8

Conclusions and Future work

We are able to obtain optimal pulse sequence for sparse target unitary, using TRPO. This solution can be used to devise efficient quantum gates, such that, they can be used to build the basic component of modular quantum computer. Given the complexity of the problem, extending the solution to higher qubit systems is a bit non-intuitive. Few aspects that can be looked upon are discussed below,

8.1 Extending to higher qubit systems

The problem consists of two parts, the quantum computing part and the machine learning part. The former involves the actual dynamics that drives the system which is not classical and simulation of the dynamics on classical computer with equivalent performance is not straightforward. If provided, that the time-evolution and fidelity calculations can be done in efficient way, then the above mentioned methods can be used to provide solutions for higher qubit systems using deeper neural networks after sufficient training.

As we have seen in the three-qubit transmon system, the physical space is 4^3 -dimensional and we project this into 20×20 space and then into 8×8 computational space. This process would become tedious while trying to extend this approach to higher dimensions. The next step would be to try to figure out a scalable solution to convert the physical space to computational space in realizable time. One way is to separate out the fidelity calculation part and RL part. That is, for higher dimensions, the computation of reward is too high due to matrix multiplications of larger matrices. If we can use a quantum system to make our measurements, the computational cost will become realizable in practical time, even for higher dimensions. Then we can apply a hybrid deep RL method to optimize it.

8.2 Utilizing the solution to build scalable practical machines

Solving higher qubit systems can also be seen as, solving bigger Rubik's cube when you have methods and solutions for solving smaller Rubik's cube. The approach will be similar, like, trained agents which can solve the smaller problem will have the intuition about the core issues behind solving the problem embedded as features in the network and a group of such trained agents together can associate multitude of such diverse features to get a better grasp on the landscape of the problem to solve the larger problem.

Given the above restriction on scalability to higher dimensions, bootstrapping a quantum computer to leverage the computational advantage might be another solution to bring us closer to realization of quantum computers. As this is a search problem, another way to solve this is to create modified Grover's search to search the computational space, by devising the appropriate oracle. Devising the oracle is the bottleneck here. The scalable hybrid model, mentioned in 7.3 can also be used to design optimizer for practical machines. More details on design in section A.2

Appendix A

Software tool-kit

The methods mentioned so far are diverse in nature, varying from gradient based methods to deep neural networks with fully automated complex architecture. Rather than having a different implementation for each and every new physical system and algorithm that is being used, there is scope for designing modular units to make the code more versatile.

A.1 Design of the model

Before delving into the design, there are few observations to be made. As computation is defined as transformation of given input into desired output by following certain steps, here, the input is a set of un-optimized random pulse sequences and the desired output is set of better optimized pulse sequences. The machine learning algorithm requires data to be provided as input. Gradient based optimizers are sensitive to initial sequence, and few other machine learning algorithms are in-sensitive to the start sequence. The idea behind designing these modules is to use the optimizers as black box and leverage the sensitivity towards the starting sequence.

A.1.1 Components

The modules in the tool-kit should be re-usable and easy to modify. The core solution for the given control problem consists of three main parts, firstly, the simulation of underlying physical system to be controlled, the next part is the controller, the algorithm or method used, and lastly, the visualization, where the solutions can be analyzed.

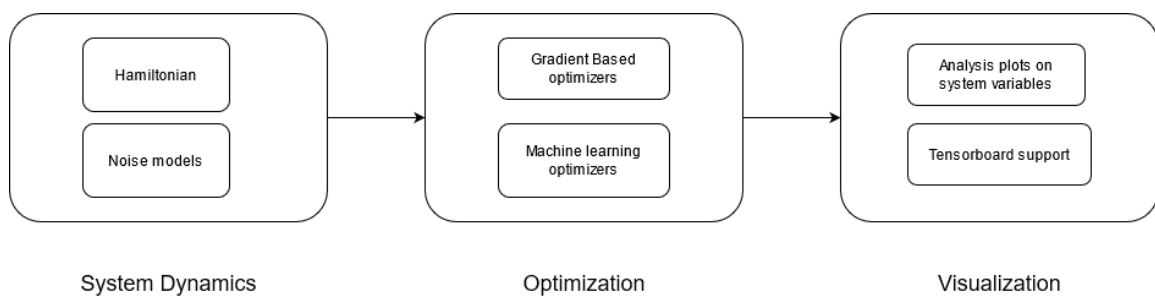


FIGURE A.1: Outline of modular units with direction of flow of the data

A.2 Implementation details

Each of the components are designed in such a way that sub-components of each component can be concatenated in any order the user desires. But each component at higher level should follow a definitive pattern. The machine learning optimizers are built using tensorflow, system dynamics is built using QuTiP, RL optimizers using stable-baselines.

The system dynamics module consists of the implementations of the underlying physical systems. The Optimization module consists of SuSSADE, naive Differential evolution gradient optimizers, along with the machine learning optimizer build over the neural networks as desired by the user for the naive deep neural network architecture and TRPO, PPO based reinforcement learning optimizer.

The visualization module consists of tensorboard support for visualizing logs corresponding to machine learning optimizers and most frequently used plots of variation of fidelity over time, variation system states over time, variation of pulse sequence and comparison across set of implemented methods.

The hybrid model as mentioned in section 7.3, is actually implemented as follows. The system dynamics module provides us with an abstraction to obtain fidelity and other relevant metrics for the optimizer. This is provided as input for the gradient-based optimizer, which is iteratively proceeds till the convergence is achieved or maximum number of iterations are reached. The whole trace during this process is captured and is passed to machine learning optimizer. So, the input for the ML optimizer is this data and the optimizer trains over this new data and at the end, the output from this ML optimizer is a set of random pulse sequence in the user-defined dimensions which in-turn be given as an input, expecting a better optimized pulse sequence from the optimizer which can be recursively provided as input for the gradient based optimizer until convergence is achieved.

Bibliography

- [1] Zheng An and D. L. Zhou. *Deep Reinforcement Learning for Quantum Gate Control*. URL: <https://arxiv.org/pdf/1902.08418.pdf>.
- [2] Bram Bakker. *Reinforcement Learning with long short term Memory*. URL: <https://papers.nips.cc/paper/2001/file/a38b16173474ba8b1a95bcb30d3b8a5-Paper.pdf>.
- [3] Luca Innocenti; Leonardo Banchi; Alessandro Ferraro; Sougato Bose; and Mauro Paternostro. *Supervised learning of time-independent Hamiltonians for gate design*. URL: <https://iopscience.iop.org/article/10.1088/1367-2630/ab8aaf>.
- [4] Petre Birtea; Ioan Căsu; Dan Comanescu. *Constraint optimization and SU(N) quantum control landscapes*. URL: <https://arxiv.org/abs/2103.11132v2>.
- [5] Tanay Roy; Suman Kundu; Madhavi Chand; Sumeru Hazra; N. Nehra; R. Cosmic; A. Ranadive; Meghan P. Patankar; Kedar Damle; and R. Vijay. *Implementation of Pairwise Longitudinal Coupling in a Three-Qubit Superconducting Circuit*. URL: <https://journals.aps.org/prapplied/abstract/10.1103/PhysRevApplied.7.054025>.
- [6] J. Ghosh E. Zahedinejad and B. C. Sanders. *Designing high-fidelity single-shot three-qubit gates: a machine-learning approach*. *Physical Review Applied*, 6(5): 054005, 2016.
- [7] Elija Perrier; Dacheng Tao; Chris Ferrie. *Quantum Geometric Machine Learning for Quantum Circuits and Control*. URL: <https://arxiv.org/abs/2006.11332>.
- [8] J. Gallardo G. Dattoli and A. Torre. *Time-ordering techniques and solution of differential difference equation appearing in quantum optics*. *Journal of mathematical physics*, 27(3):772–780, 1986.
- [9] Wen-Long Ma; Shruti Puri; Robert J. Schoelkopf; Michel H. Devoret; S. M. Girvin; and Liang Jiang. *Quantum control of bosonic modes with superconducting circuits*. URL: <https://arxiv.org/abs/2102.09668>.
- [10] Kentaro Heya and Naoki Kanazawa. *Cross cross resonance gate*. URL: <https://arxiv.org/pdf/2103.00024.pdf>.
- [11] Adam Rothman; Tak-San Ho; and Herschel Rabitz. *Observable-preserving control of quantum dynamics over a family of related systems*. URL: <https://journals.aps.org/prapplied/abstract/10.1103/PhysRevA.72.023416>.
- [12] S. Lloyd and S. Montangero. *Information Theoretical Analysis of Quantum Optimal Control*. URL: <https://journals.aps.org/prl/abstract/10.1103/PhysRevLett.113.010502>.
- [13] Seth Lloyd and Reevu Maity. *Efficient implementation of unitary transformations*. URL: <https://arxiv.org/abs/1901.03431>.
- [14] Xiaoting Wanga; Michele Allegrac; Kurt Jacobsd; Seth Lloydb; Cosmo Lupof; and Masoud Mohsenig. *Time-optimal quantum control via differential geometry*. URL: <https://dspace.mit.edu/bitstream/handle/1721.1/115306/101180F.pdf>.
- [15] Anatoli Polkovnikov; Pankaj Mehta Marin Bukov; Alexandre G.R. Day; Dries Sels; Phillip Weinberg. *Reinforcement Learning in Different Phases of Quantum Control*. URL: <https://arxiv.org/pdf/1705.00565.pdf>.
- [16] Susanna Kirchhoff; Torsten Kebler; Per J. Liebermann; Elie Assema; Shai Machnes; Felix Motzoi; and Frank K. Wilhelm. *Optimized cross-resonance gate for coupled transmon systems*. URL: <https://journals.aps.org/prapplied/abstract/10.1103/PhysRevA.97.042348>.
- [17] Vadim Smelyanskiy Murphy Yuezhen Niu Sergio Boixo and Hartmut Neven. *Universal Quantum Control through Deep Reinforcement Learning*. URL: <https://arxiv.org/pdf/1803.01857.pdf>.
- [18] M. Nakahara and T. Ohmi. *Quantum computing: from linear algebra to physical realizations*. CRC press, 2008.
- [19] Jarrod R. McClean; Sergio Boixo; Vadim N. Smelyanskiy; Ryan Babbush; Hartmut Neven. *Barren plateaus in quantum neural network training landscapes*. URL: <https://arxiv.org/abs/1803.11173>.

- [20] *Playing Atari with Deep Reinforcement Learning*. URL: <https://www.cs.toronto.edu/~vmnih/docs/dqn.pdf>.
- [21] V. R. Krithika Priya Batra and T. S. Mahesh. *Push-Pull Optimization of Quantum Controls*. URL: <https://arxiv.org/pdf/1908.06283.pdf>. (accessed: 01.09.2016).
- [22] Yaser Keneshloo; Tian Shi; Naren Ramakrishnan; Chandan K. Reddy. *Deep Reinforcement Learning for Sequence-to-Sequence Models*. URL: <https://arxiv.org/abs/1805.09461>.
- [23] Nelson Leung; Mohamed Abdelhafez; Jens Koch; David I. Schuster. *Speedup for quantum optimal control from automatic differentiation based on graphics processing units*. URL: <https://arxiv.org/abs/1612.04929>.
- [24] Thomas Propson; Brian E. Jackson; Jens Koch; Zachary Manchester; David I. Schuster. *Robust Quantum Optimal Control with Trajectory Optimization*. URL: <https://arxiv.org/abs/2103.15716>.
- [25] Kagan Tumer Shauharda Khadka. *Evolution-Guided Policy Gradient in Reinforcement Learning*. URL: <https://papers.nips.cc/paper/7395-evolution-guided-policy-gradient-in-reinforcement-learning.pdf>.
- [26] V. V. Shende and I. L. Markov. *On the cnot-cost of toffoli gates*. *arXiv preprint*. arXiv:0803.2316, 2008.
- [27] Robert C. Holte; Ariel Felner; Guni Sharon; Nathan R. Sturtevant. *Bidirectional Search That Is Guaranteed to Meet in the Middle*. URL: <https://people.engr.tamu.edu/guni/Papers/AAAI16-MM.pdf>.
- [28] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning*. URL: <https://mitpress.mit.edu/books/reinforcement-learning-second-edition>.
- [29] Shai Machnes; Elie Assemat; David Tannor; and Frank K. Wilhelm. *Tunable, Flexible, and Efficient Optimization of Control Pulses for Practical Qubits*. URL: <https://journals.aps.org/prl/abstract/10.1103/PhysRevLett.120.150401>.
- [30] M. H. Devoret; A. Wallraff; and J. M. Martinis. *Superconducting Qubits: A Short Review*. URL: <https://arxiv.org/abs/cond-mat/0411174v1>.
- [31] John van de Wetering. *Constructing quantum circuits with global gates*. URL: <https://arxiv.org/abs/2012.09061>.